

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 13: SDS Part 3

Instructors: Justin Yokota

Agenda

- Combinational Logic Delay
- Flip Flops and Registers
- Register Timing
- Clocks

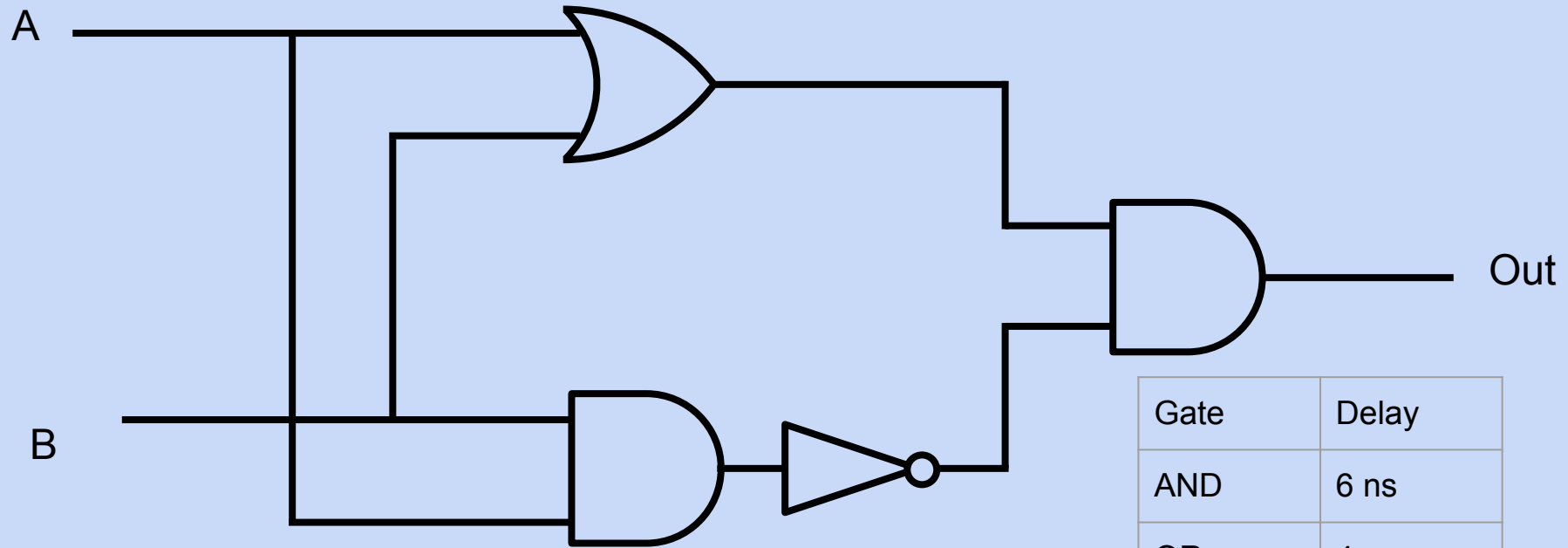
Agenda

- **Combinational Logic Delay**
- Flip Flops and Registers
- Register Timing
- Clocks

Circuit Timing

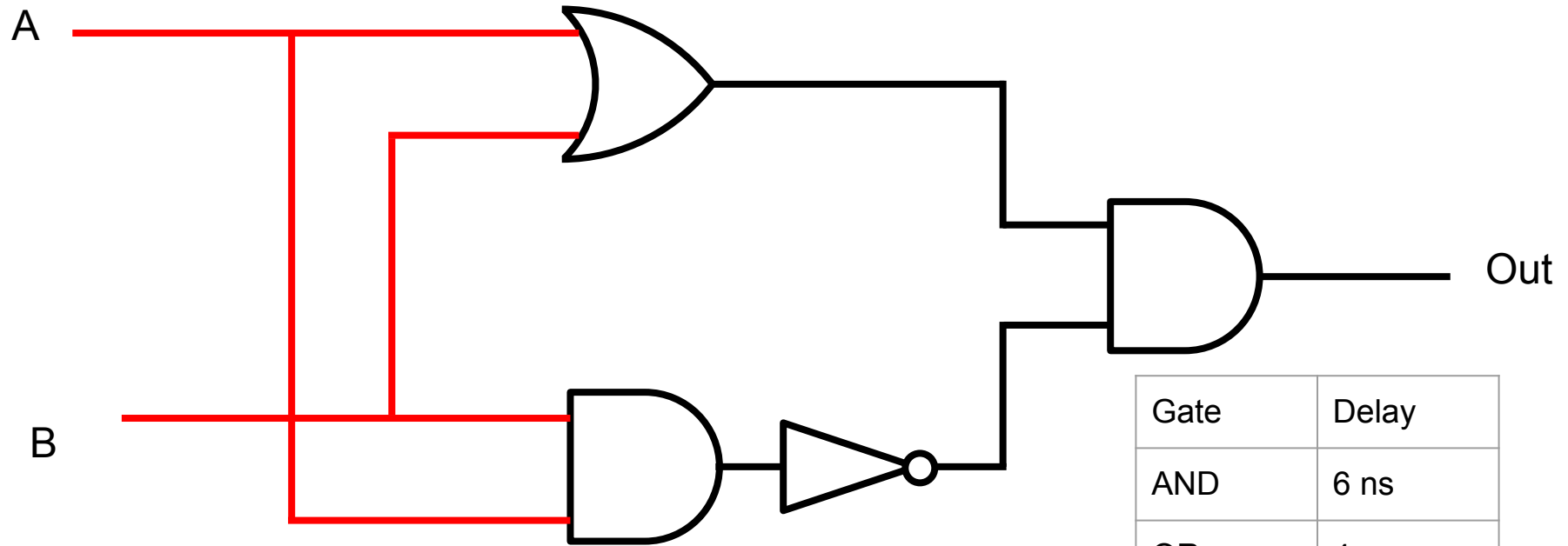
- While electricity is fast, it is not instantaneous
 - Wires transmit data at sizeable fractions of c
 - Transistor takes $\sim 10^{-11}$ seconds to "react" to gate voltage
- Every circuit component has a certain **delay**; it takes some amount of time for it to finish computing.
 - Simplification: Output of component is held steady until delay elapses, at which point output is set to new value
- For a circuit to compute properly, we can't change inputs until output is fully computed and stable.

At time 0, A and B change. When does Out change?



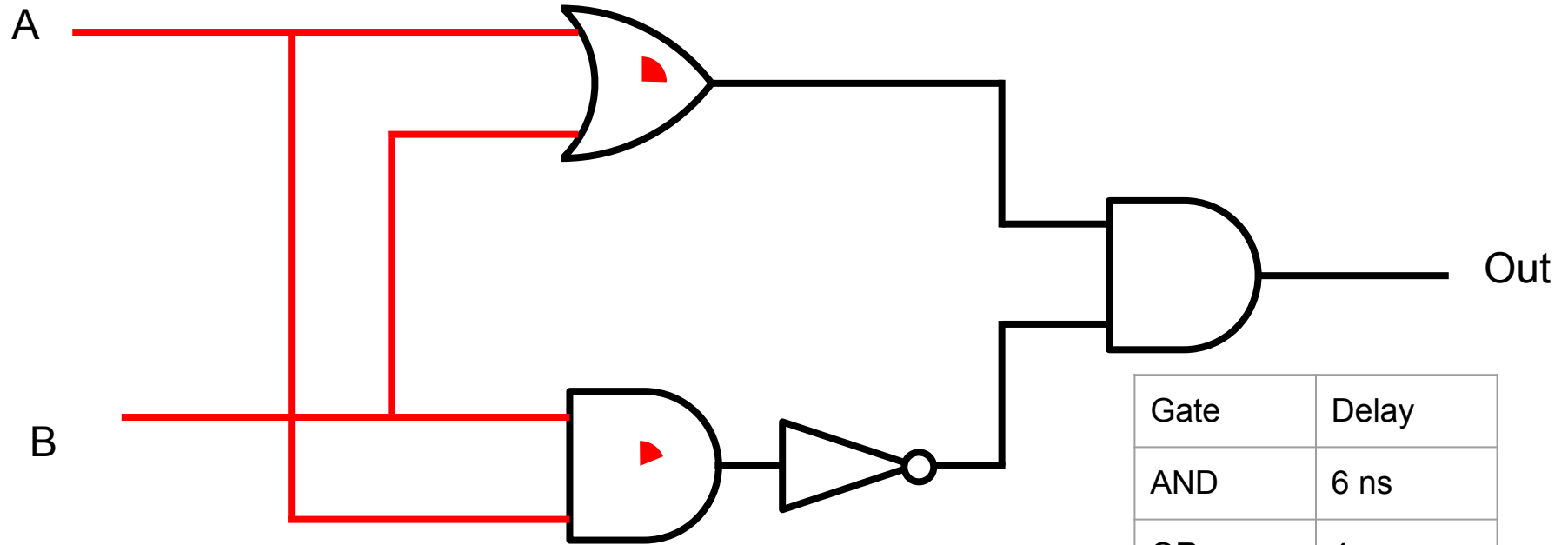
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 0 ns



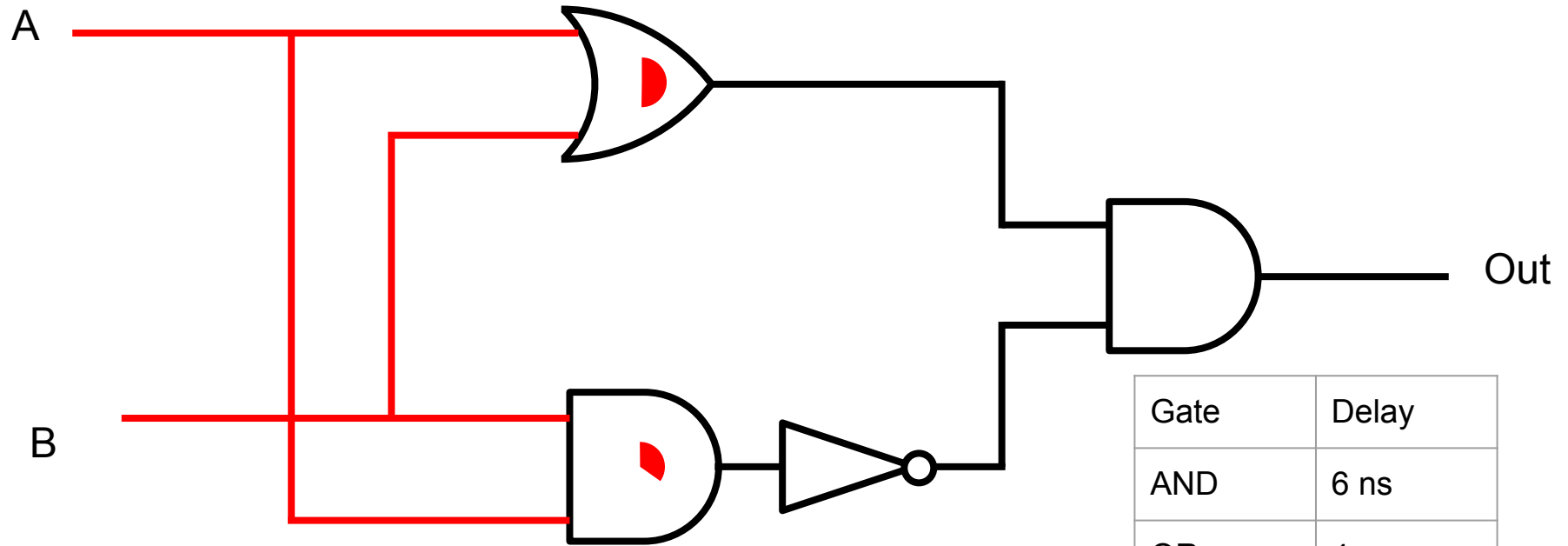
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 1 ns



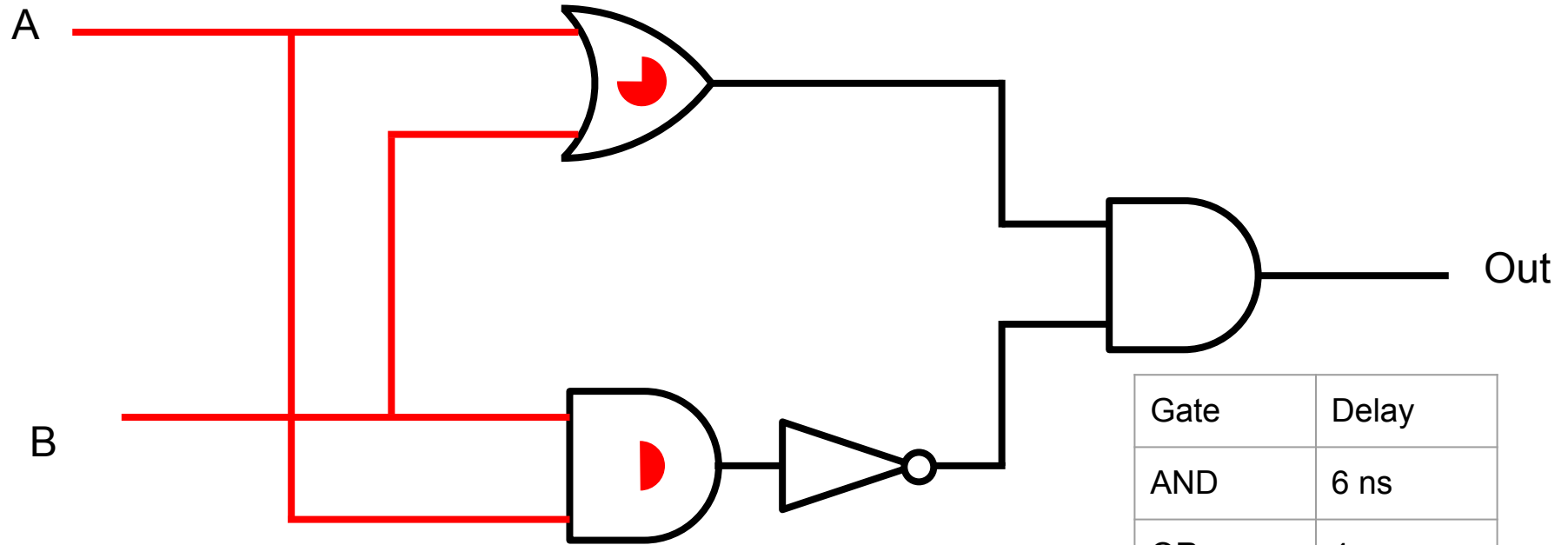
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 2 ns



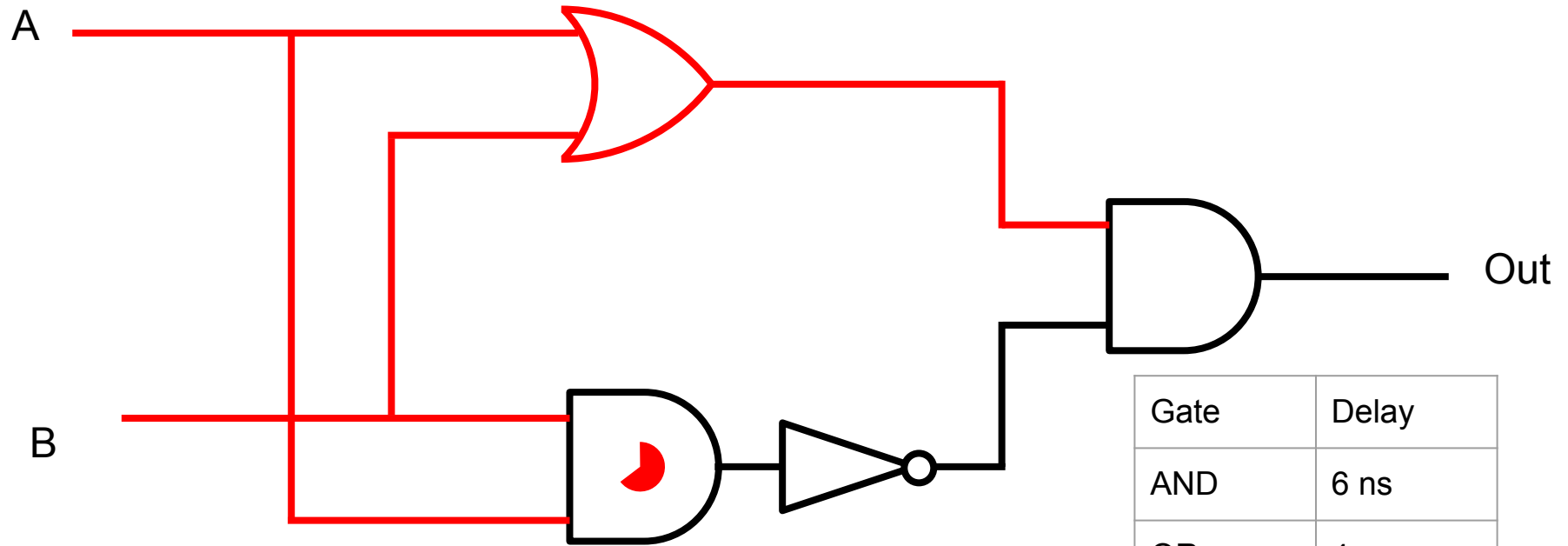
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 3 ns



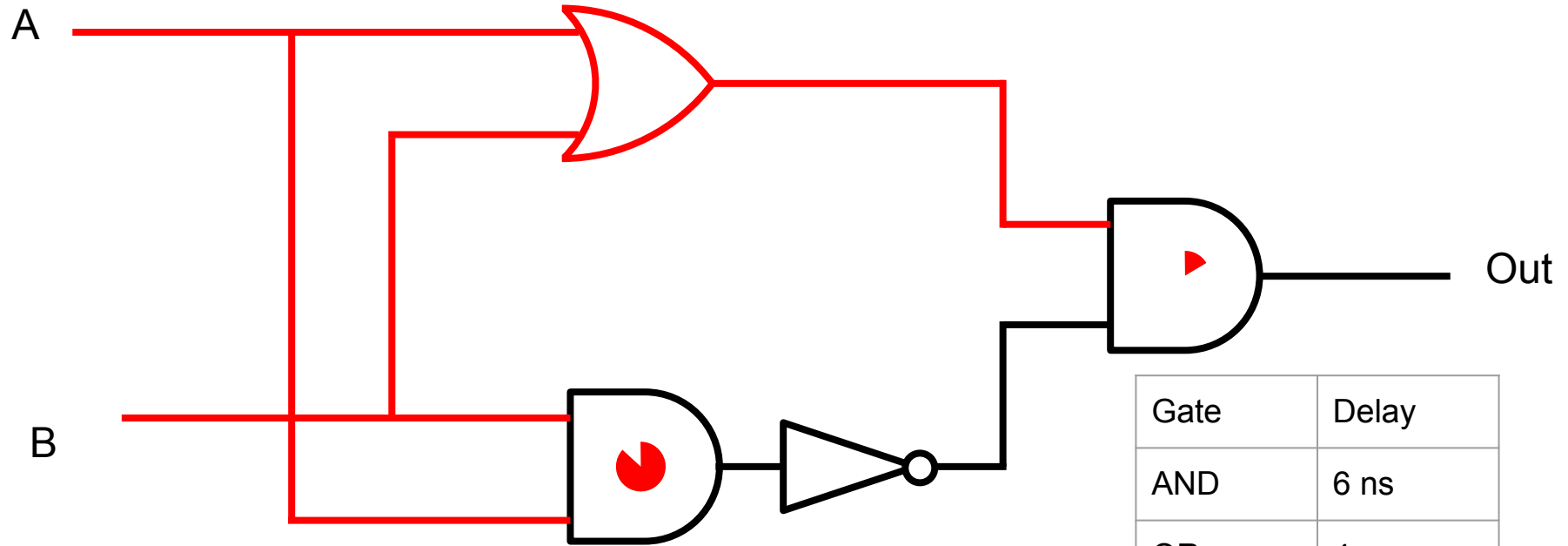
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 4 ns



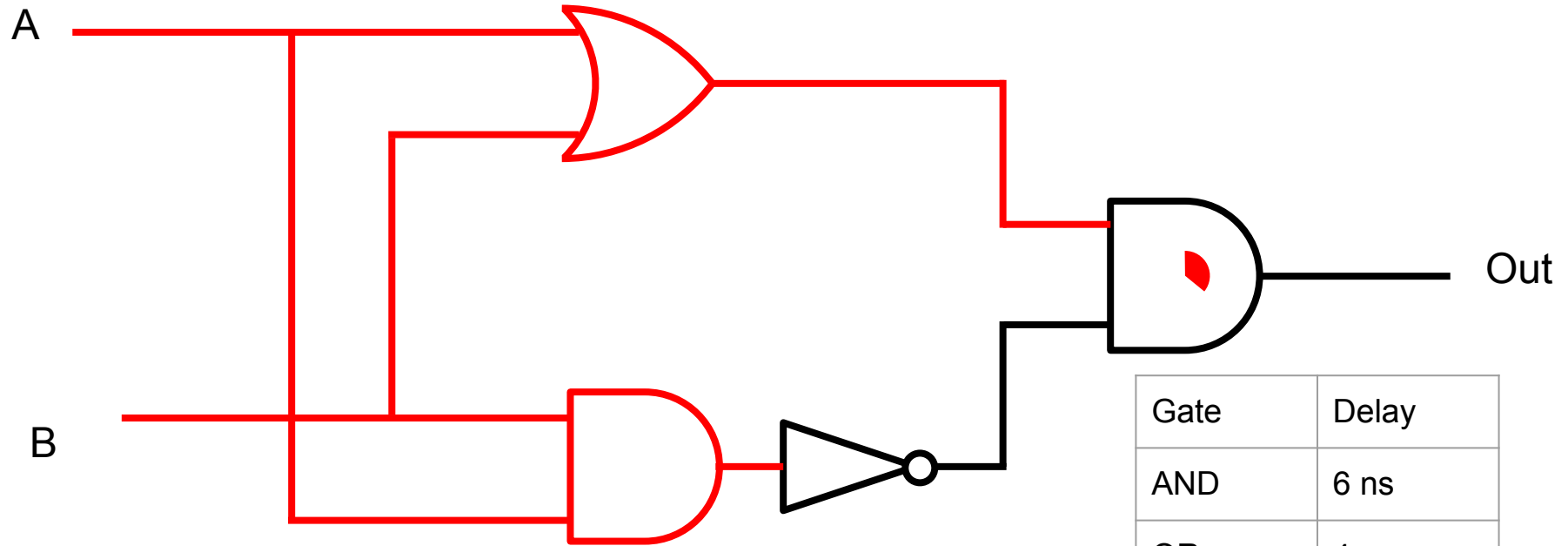
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 5 ns



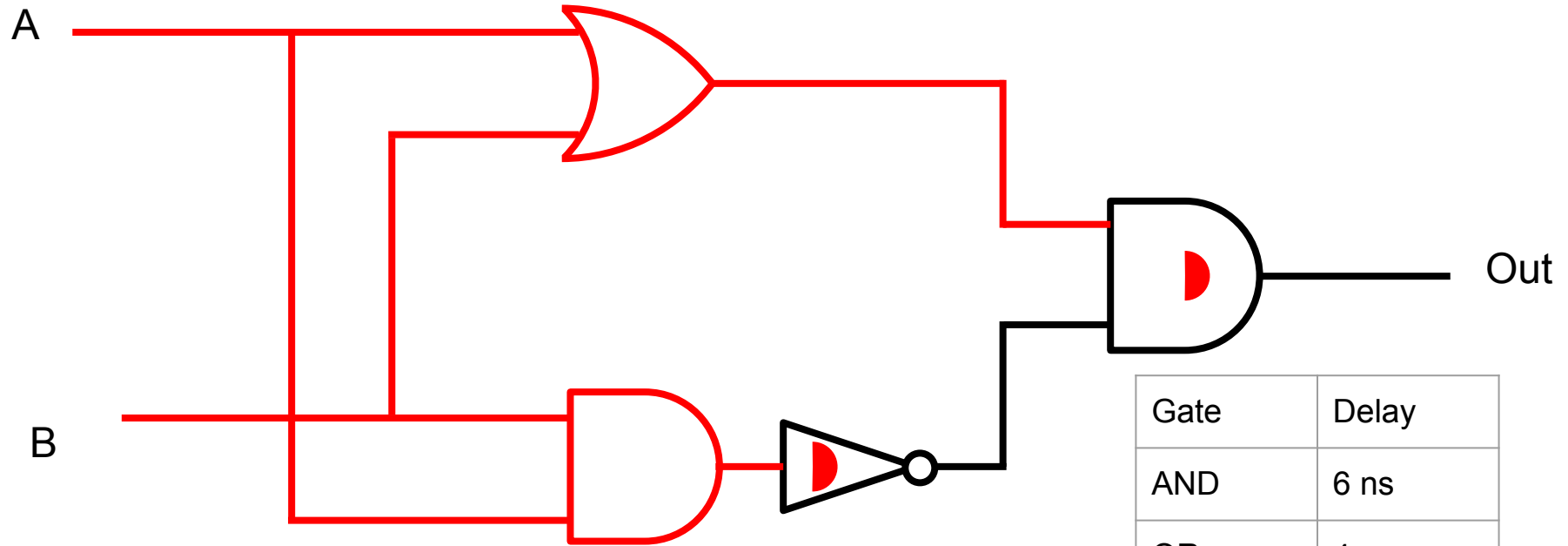
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 6 ns



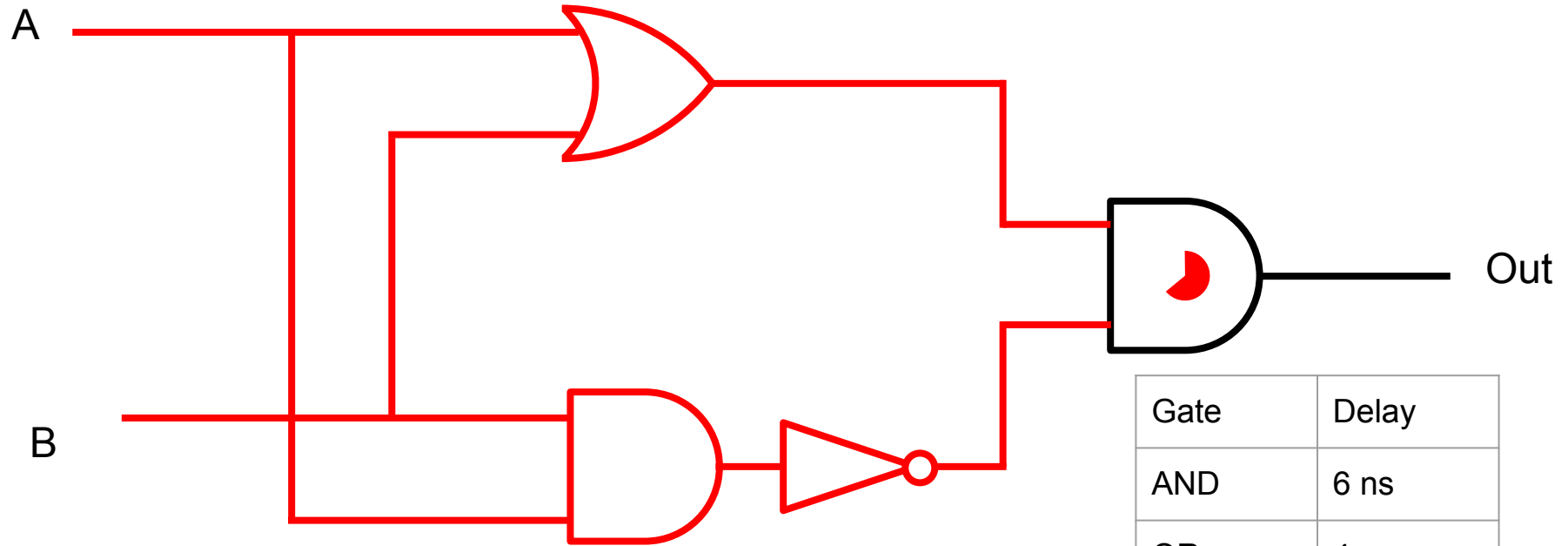
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 7 ns



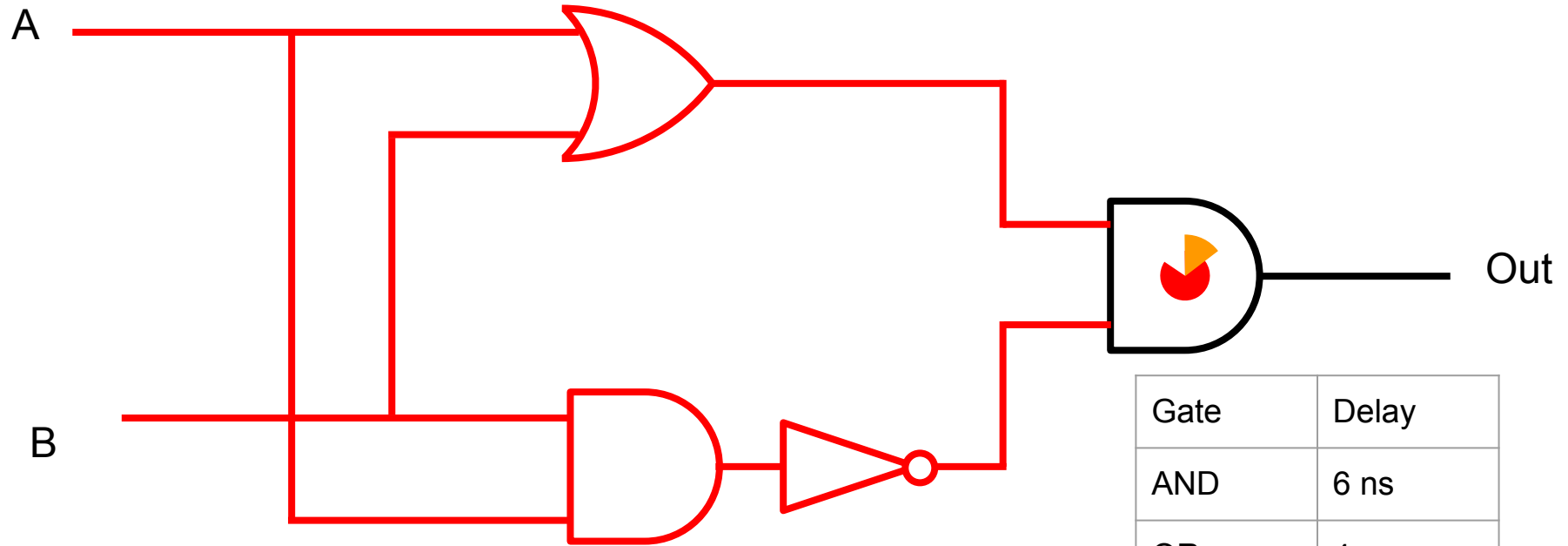
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 8 ns



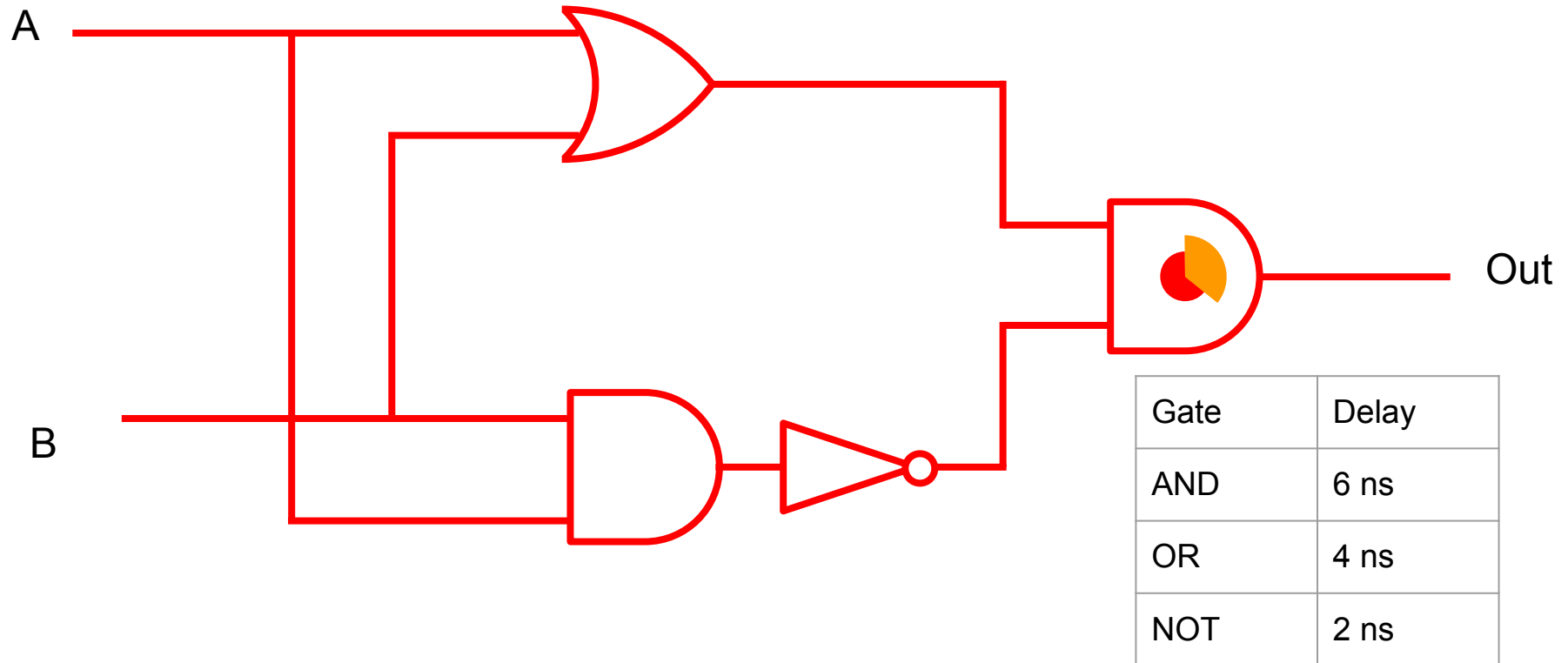
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 9 ns

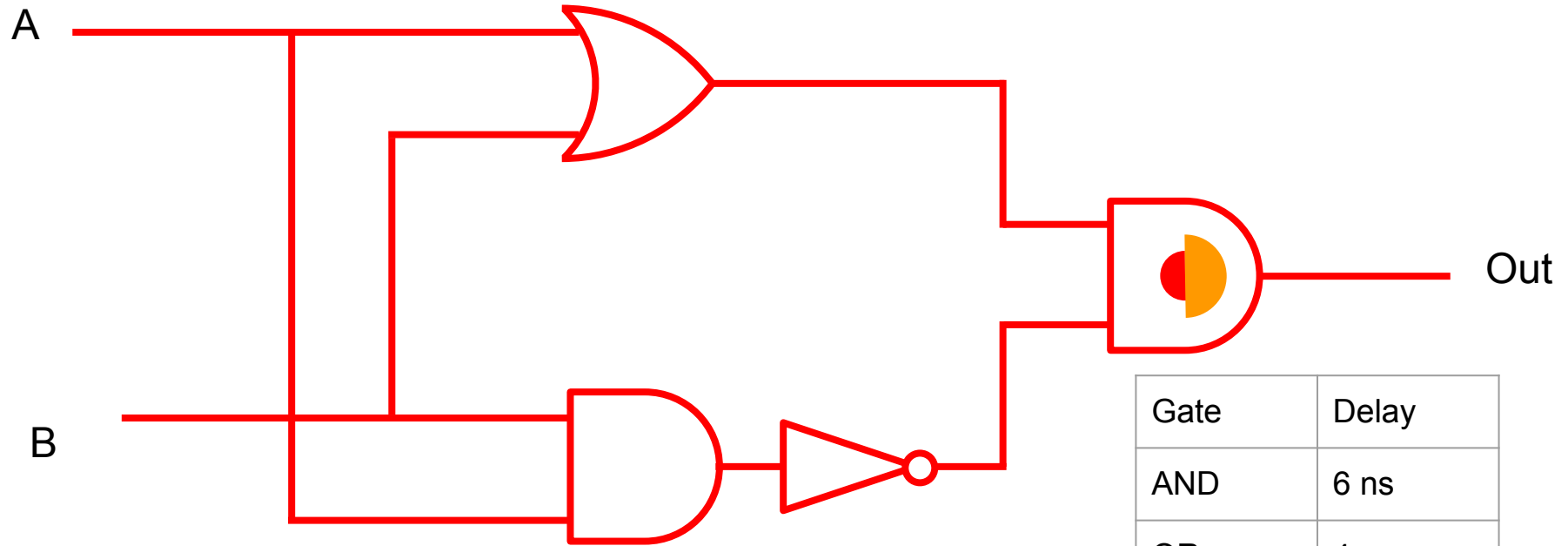


Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 10 ns: Out changes for the first time

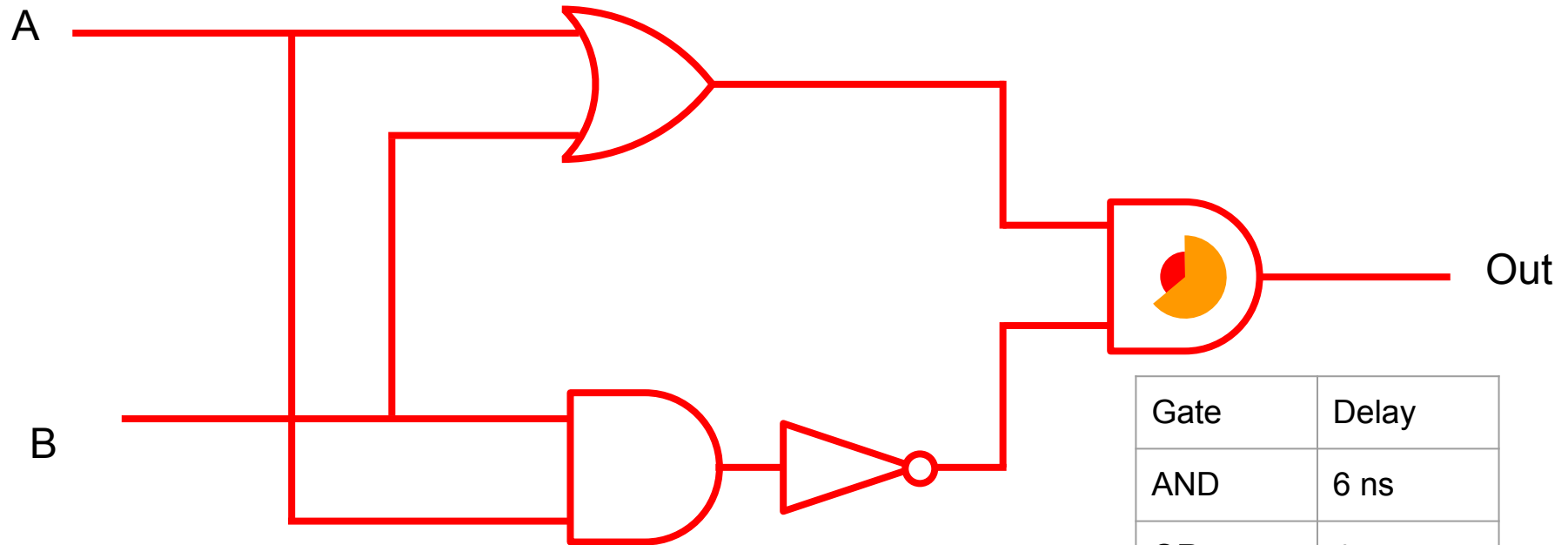


Time: 11 ns



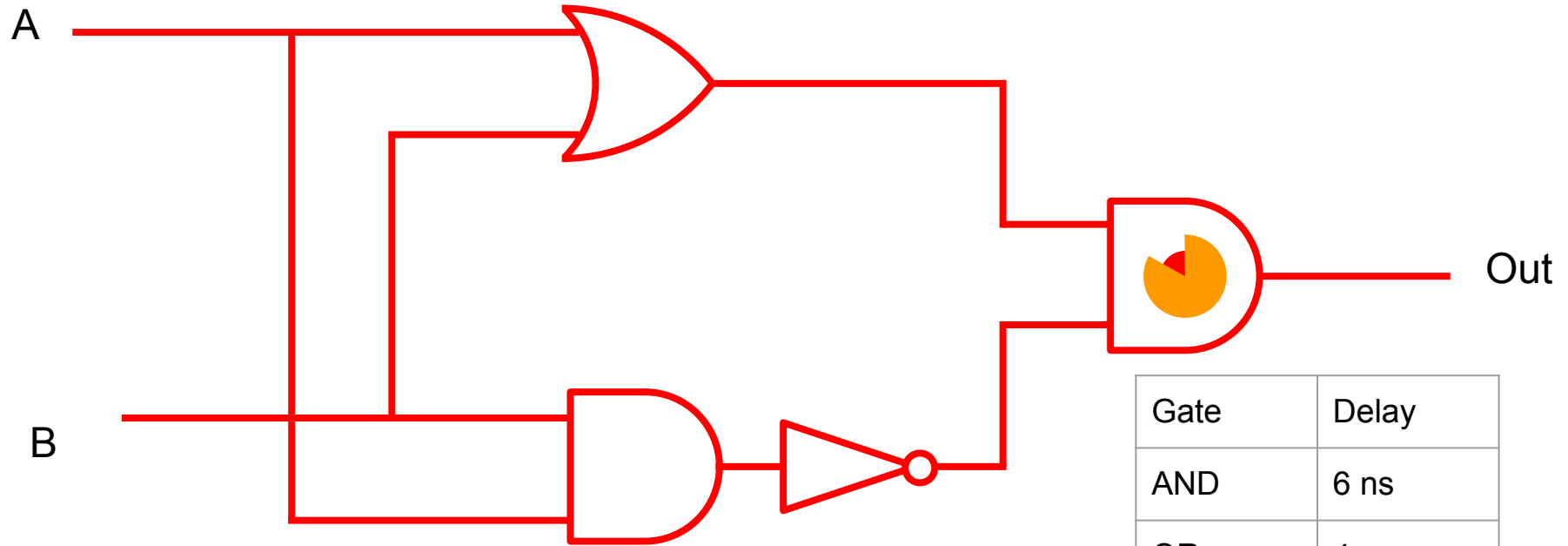
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 12 ns



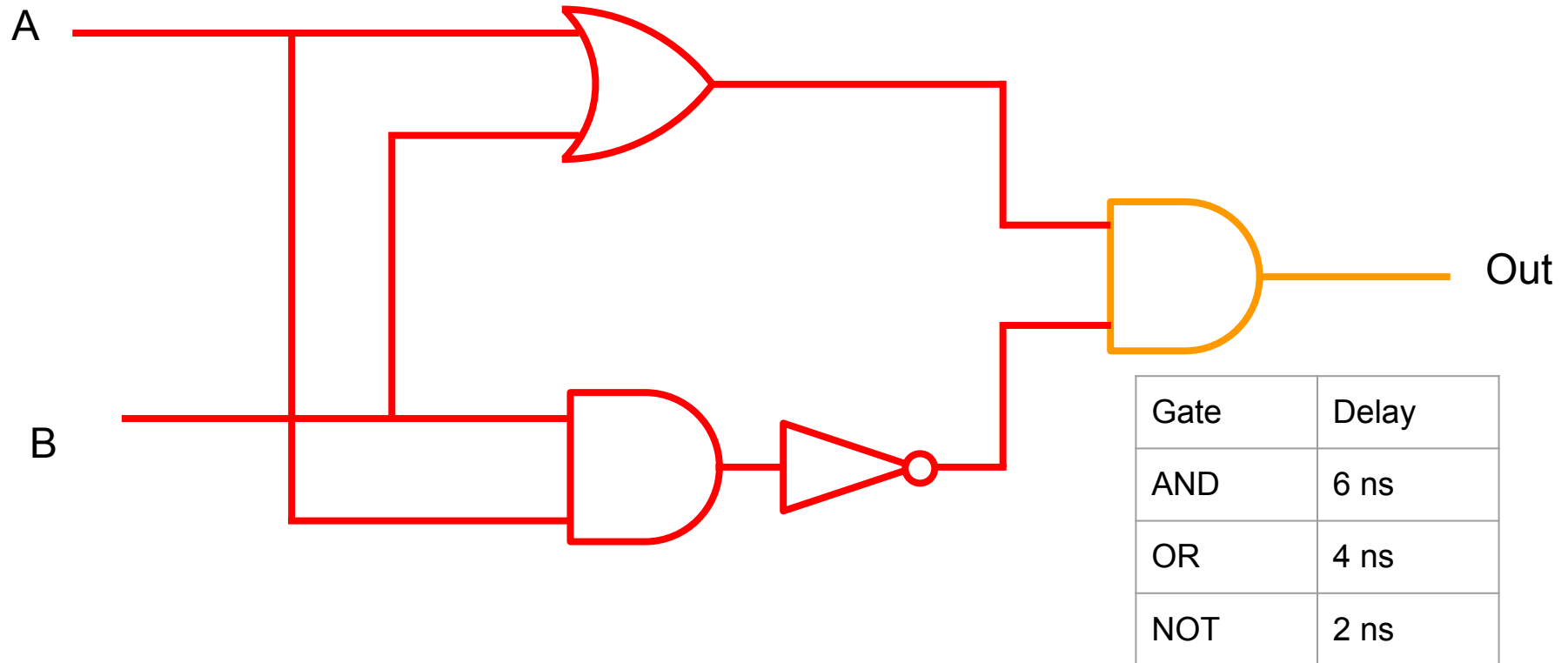
Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 13 ns



Gate	Delay
AND	6 ns
OR	4 ns
NOT	2 ns

Time: 14 ns: Out changes for the last time

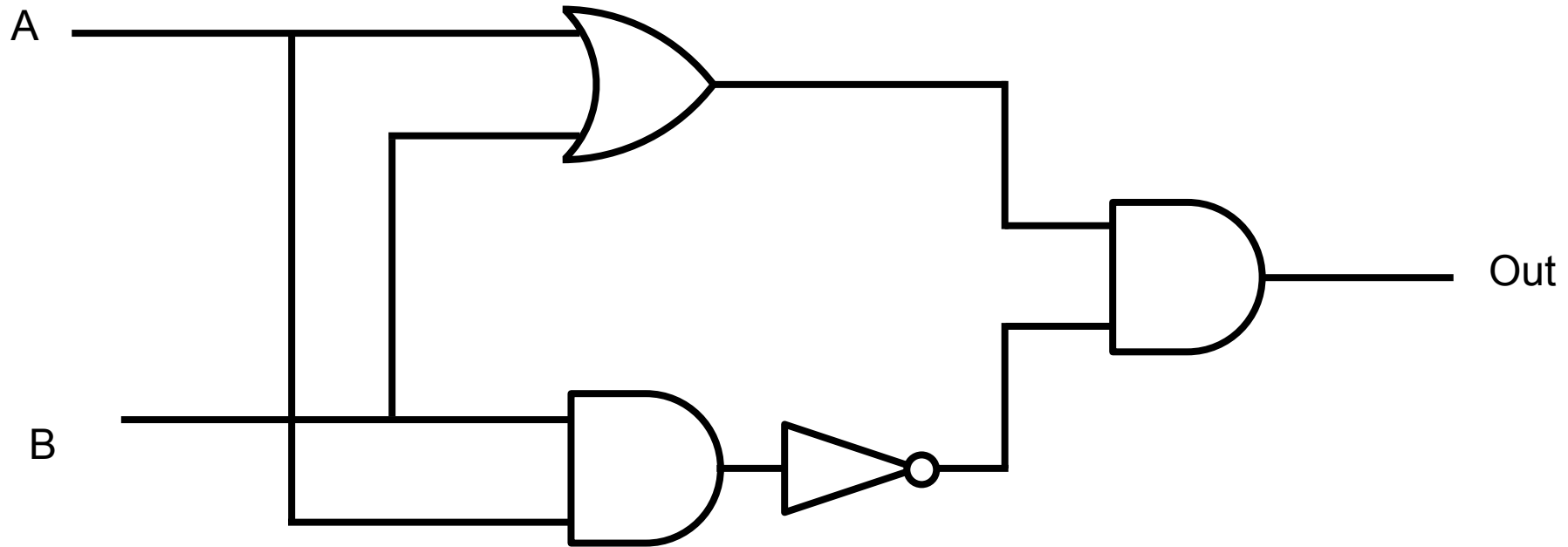


Timing Notes

- In the previous slide:
 - Time 0-10: output is stable at old value
 - Time 10: output changes to garbage value (minimum delay)
 - Time 10-14: output may or may not change?
Depending on how the AND gate behaves. Considered **undefined** value
 - Time 14+: Output is stable at the new value (maximum delay)
- To be completely safe, we must wait until the circuit is stable before using the circuit for more computation.
 - We can safely use this circuit every 14 ns.

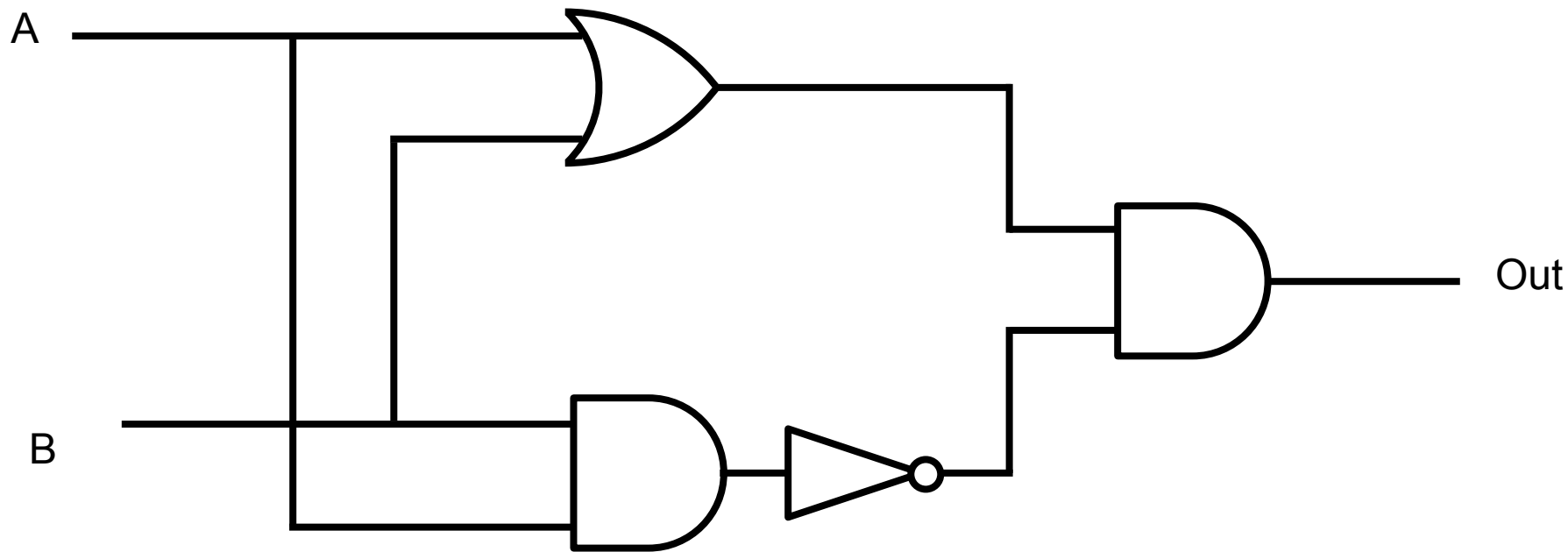
Circuit delay: 8 ns - 14 ns

Most combinational logic has a minimum delay and a maximum delay



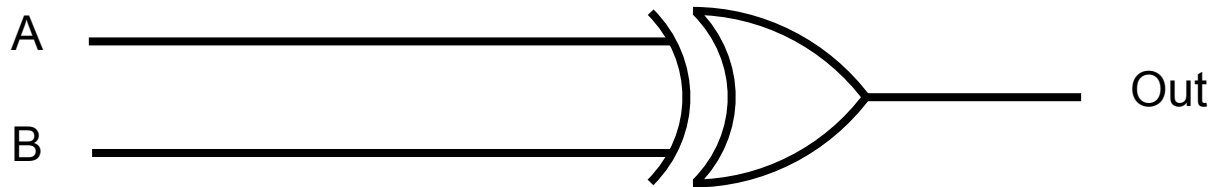
Circuit delay: 8 ns - 14 ns

Between min and max delay, output is undefined/garbage. Before/after delay, output is well-defined (old or new value)



XOR gate delay: 14 ns

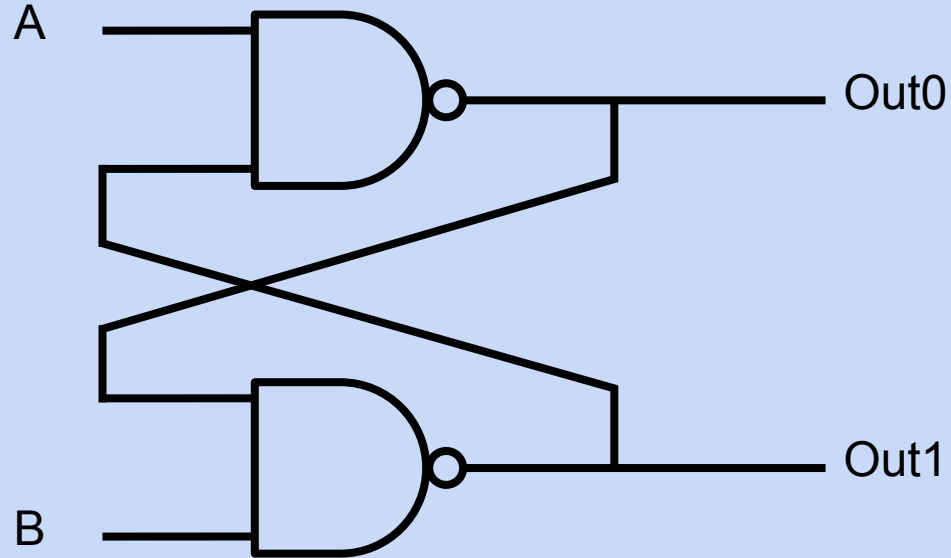
For simplicity, we assume primitive components (any component we give an explicit delay value for in a table) have min delay = max delay (in this class only).



Agenda

- Combinational Logic Delay
- **Flip Flops and Registers**
- Register Timing
- Clocks

What does this circuit do?



NAND Latch

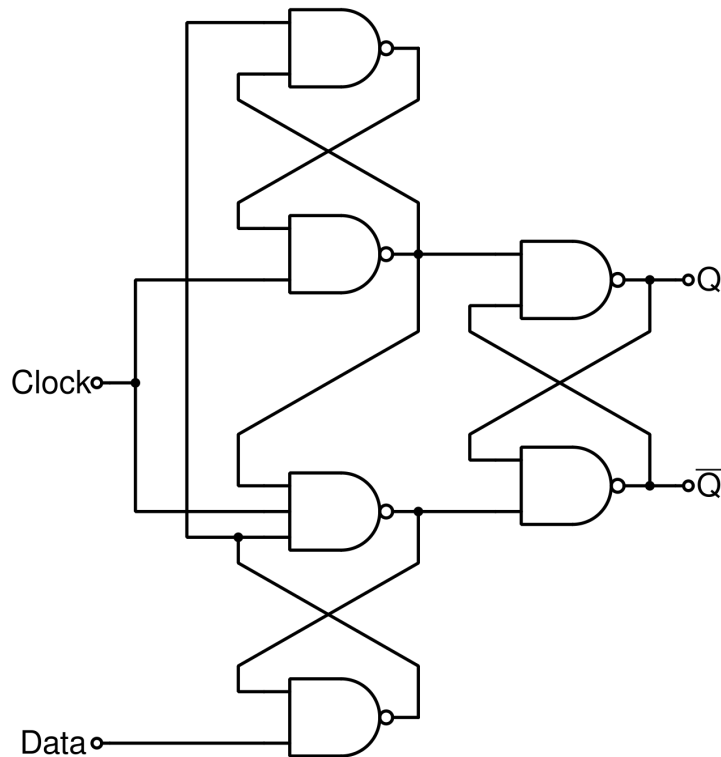
- When $A = 0$, $B = 0$, $Out0$ and $Out1 = 1$ (state 00)
- When $A = 0$, $B = 1$, $Out0 = 1$, $Out1 = 0$ (state 01)
- When $A = 1$, $B = 0$, $Out0 = 0$, $Out1 = 1$ (state 10)
- When $A = 1$, $B = 1$, things get interesting
 - If we moved from state 01 or 10, $Out0$ and $Out1$ stay stable
 - If we moved from state 00, $Out0$ and $Out1$ oscillate wildly
- Important: This circuit "remembers" the value A and B used to be as long as A and B are both 1.

State Elements

- **State Elements** are circuit components that store information
 - Needs two phases:
 - Set: Sets the value of output to a certain value
 - In latch, 10 and 01 input
 - Stable: While in this phase, output doesn't change from the value it was previously set to.
 - In latch, 11 input
- Ideally, want "set" state to be a controlled "trigger"

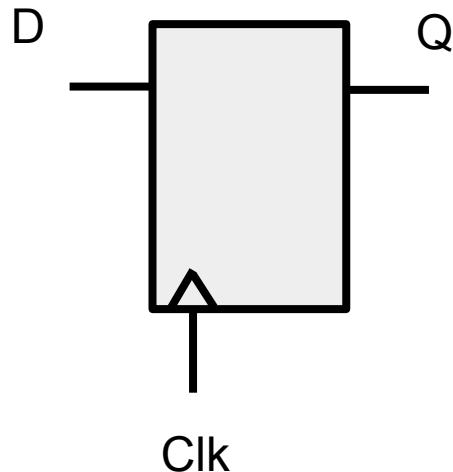
Flip-Flop

- Latch by itself reacts to its input whenever either input is 0.
- More complex structure: Flip-Flop
- Receives a clock input, and a data input
- Output reacts to data value **only when the clock changes from 0 to 1** (rising edge)



Flip-Flop

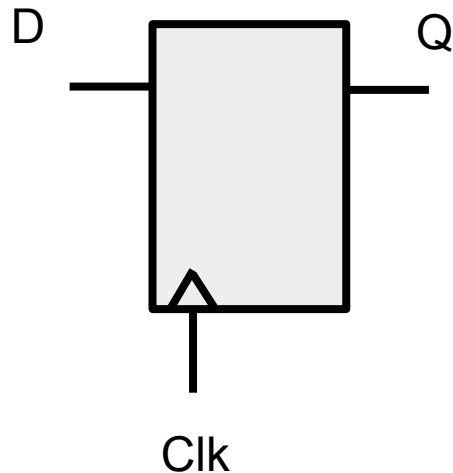
- Latch by itself reacts to its input whenever either input is 0.
- More complex structure: Flip-Flop
- Receives a clock input, and a data input
- Output reacts to data value **only when the clock changes from 0 to 1** (rising edge)



Flip-Flop

Formal behavior:

- When Clk changes from 0 to 1, set Q to the same value as D
- At all other times, ignore value in D, and maintain value of Q

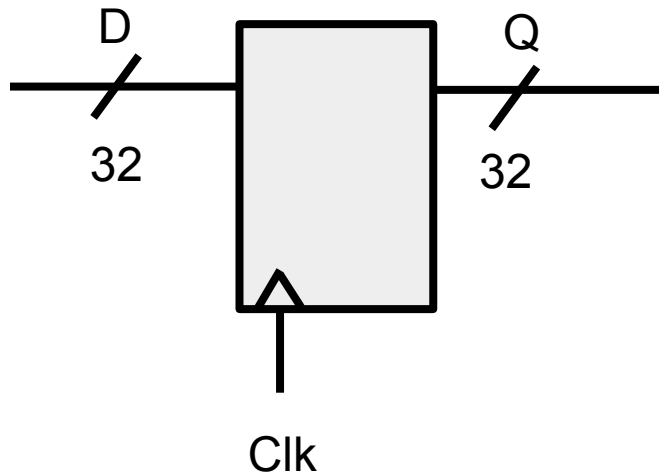


Register

32 flip-flops together make a **register**:

- When Clk changes from 0 to 1, set Q to the same value as D
- At all other times, ignore value in D, and maintain value of Q

Same behavior as flip-flop, but 32 bits at a time.



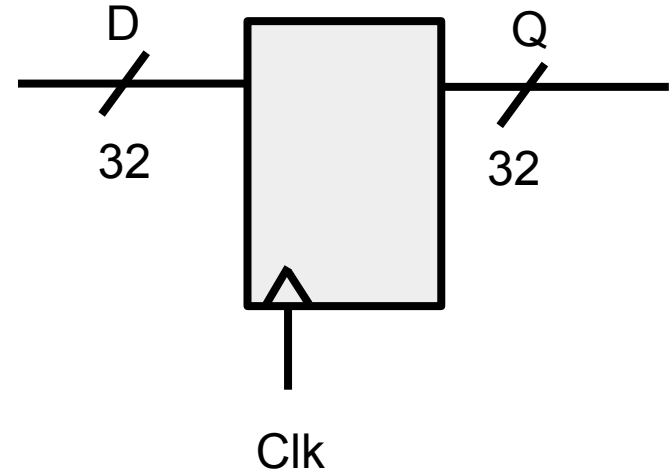
Example: Accumulator

I want to create a circuit that behaves as follows:

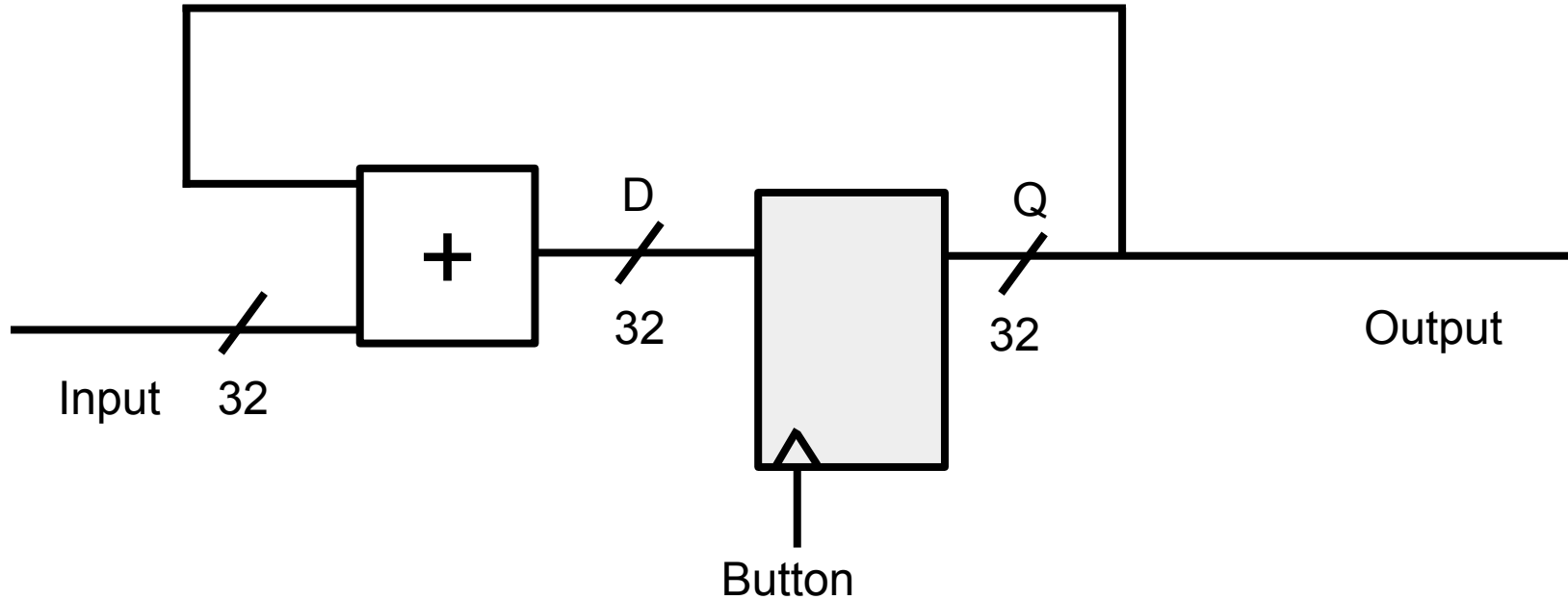
Inputs: Button, 32 bit integer

Output: 32 bit integer

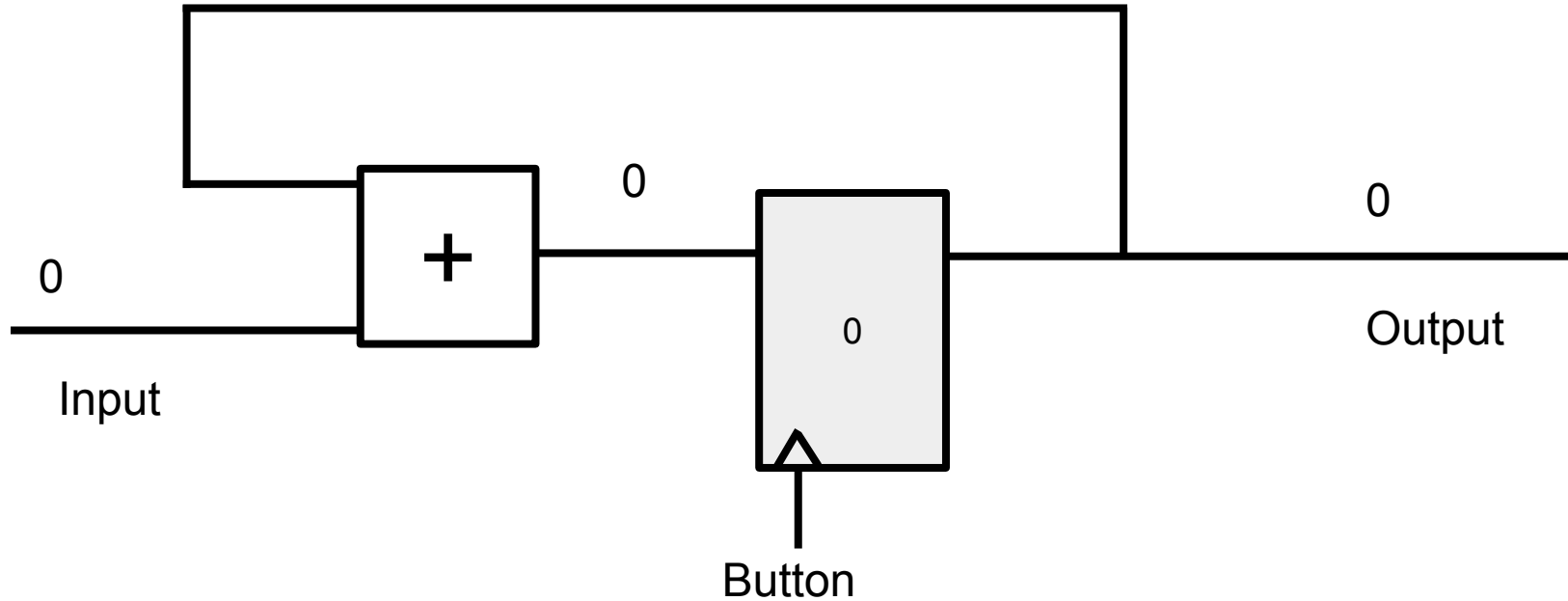
At each time step, I set the input value to a number, then I press the button. Output should be the sum of all numbers so far.



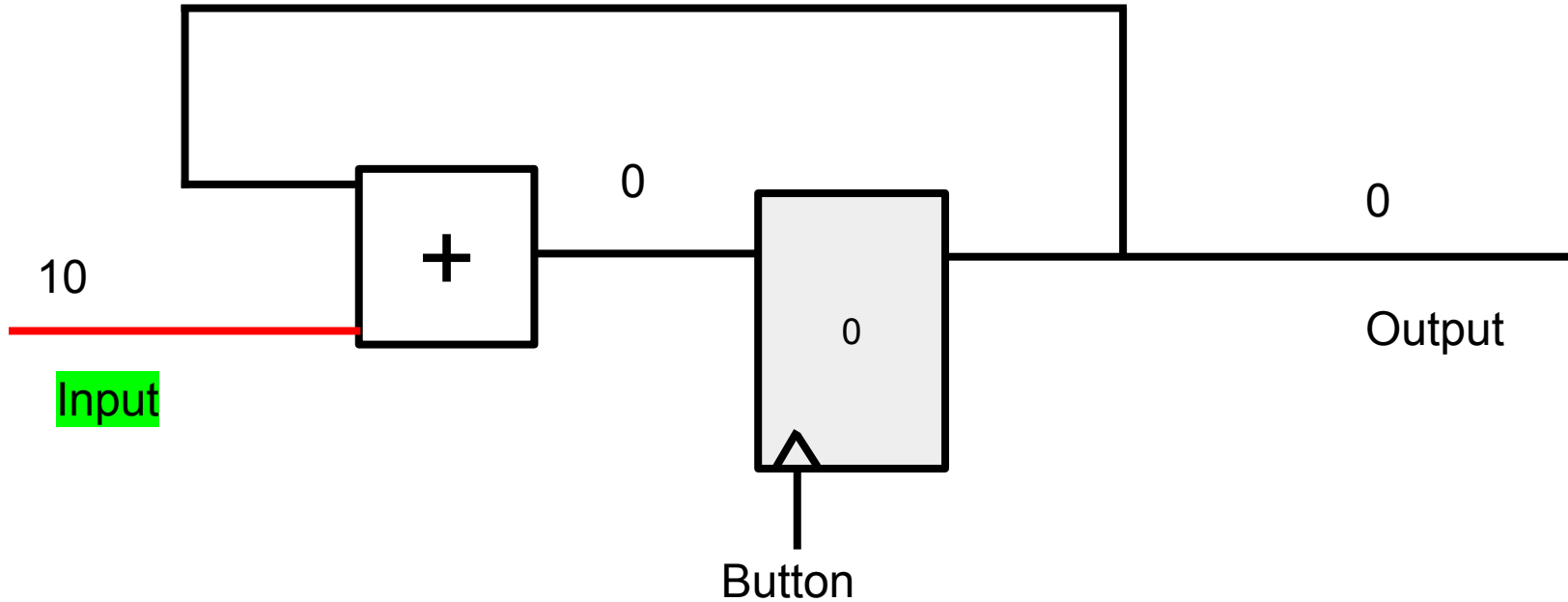
Example: Accumulator



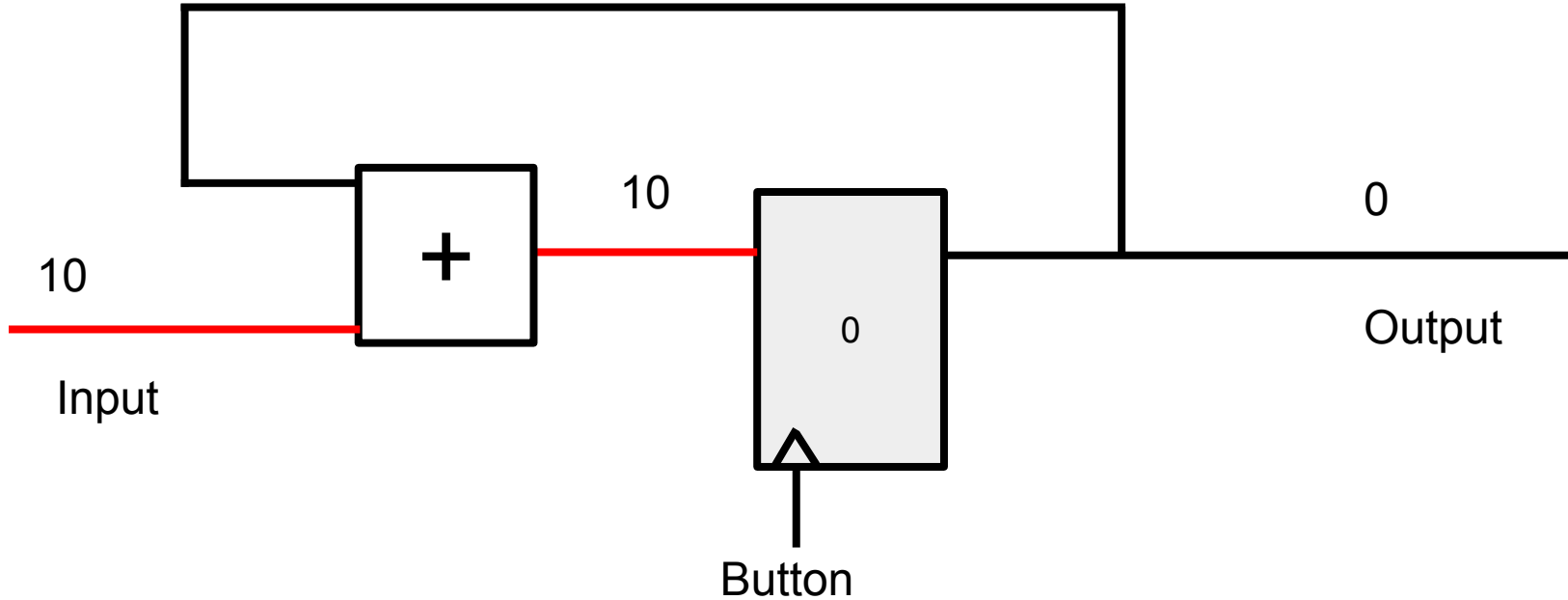
Example: Accumulator on input [10,20,30]



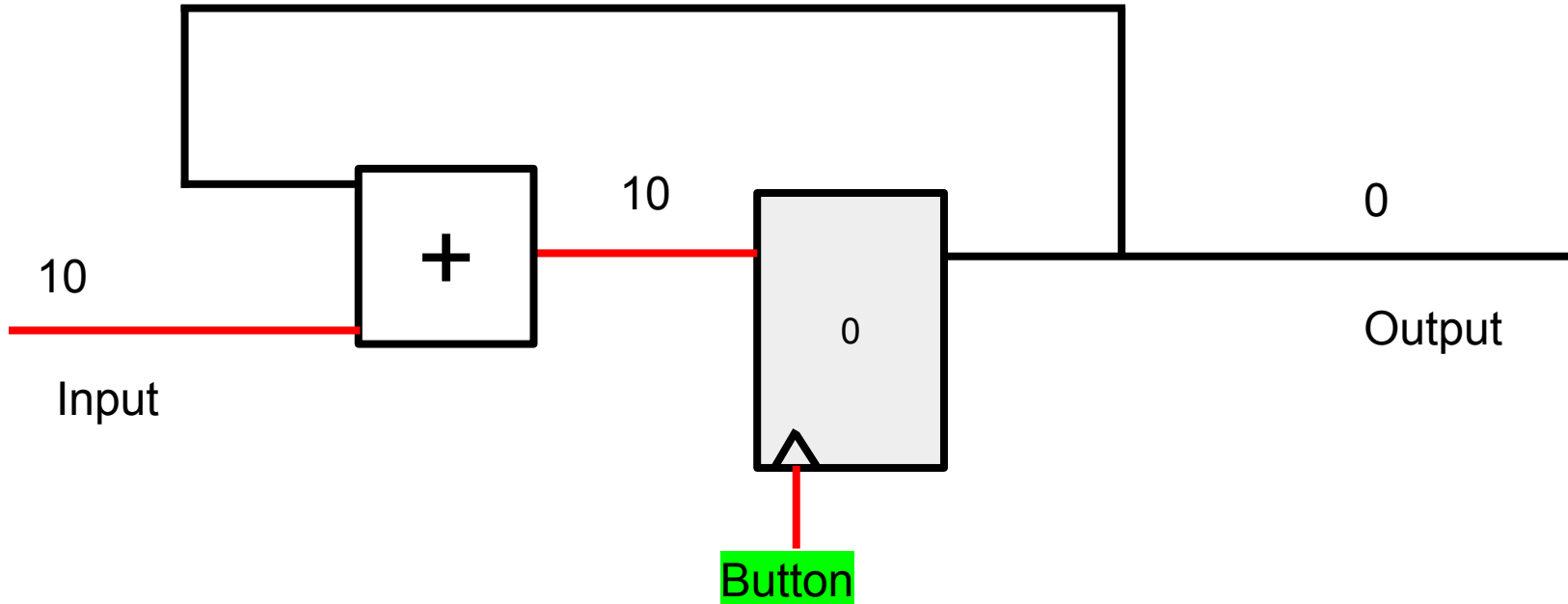
Example: Accumulator on input [10,20,30]



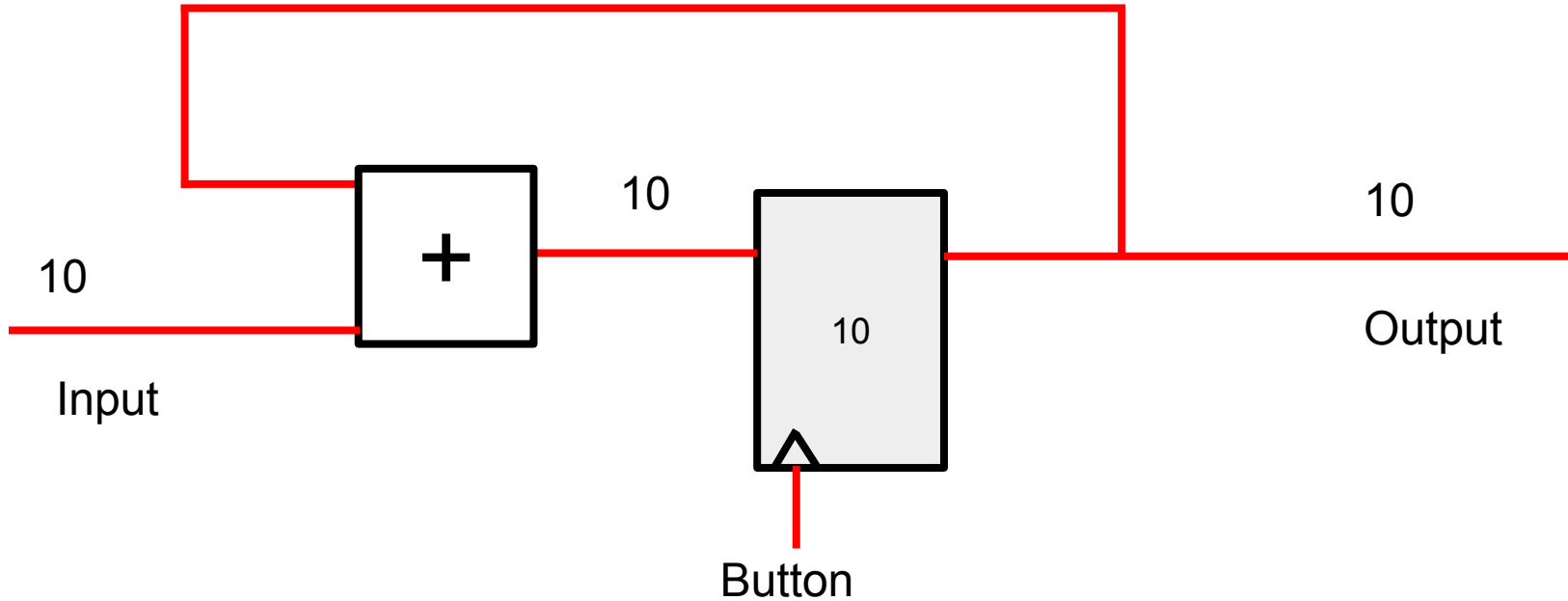
Example: Accumulator on input [10,20,30]



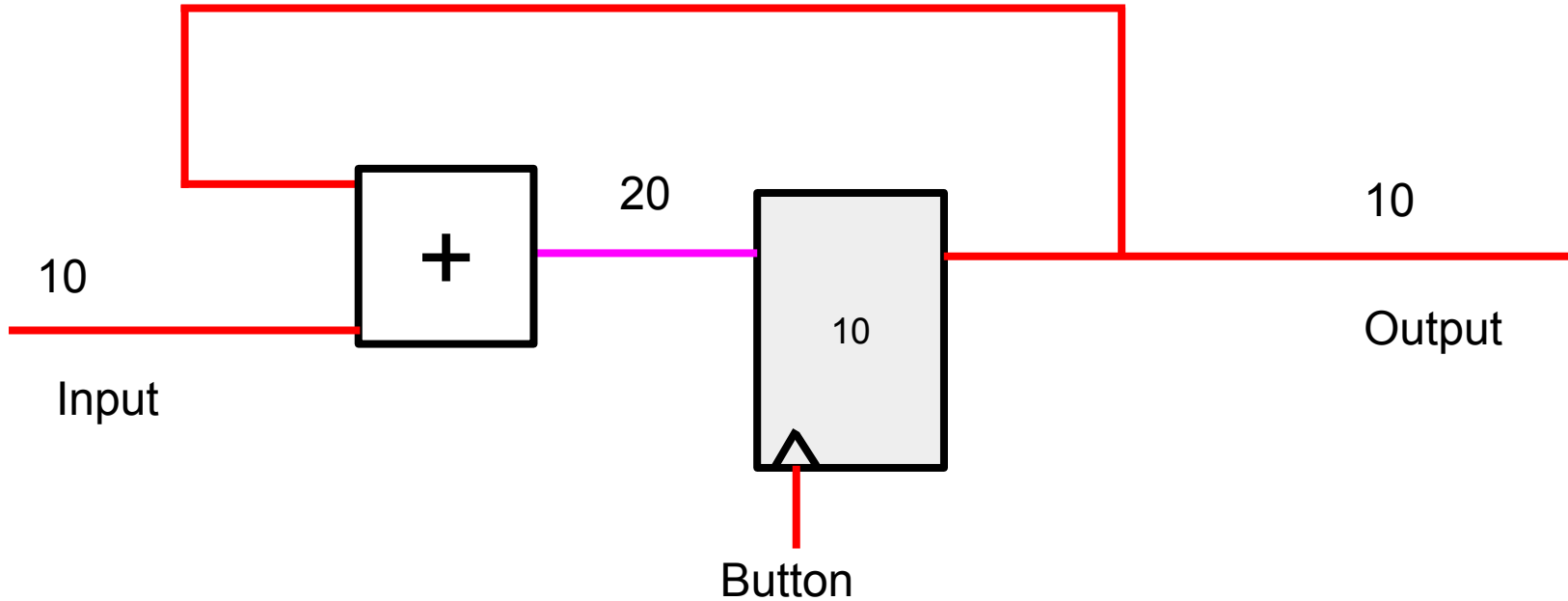
Example: Accumulator on input [10,20,30]



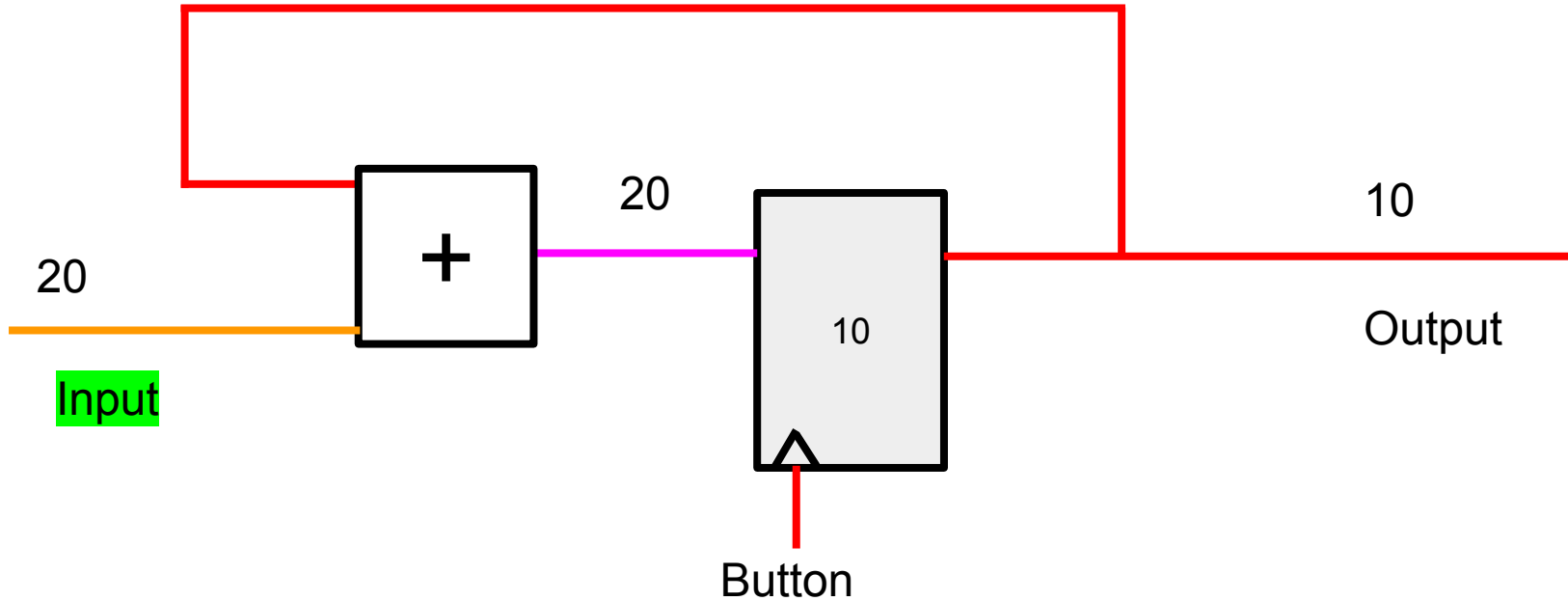
Example: Accumulator on input [10,20,30]



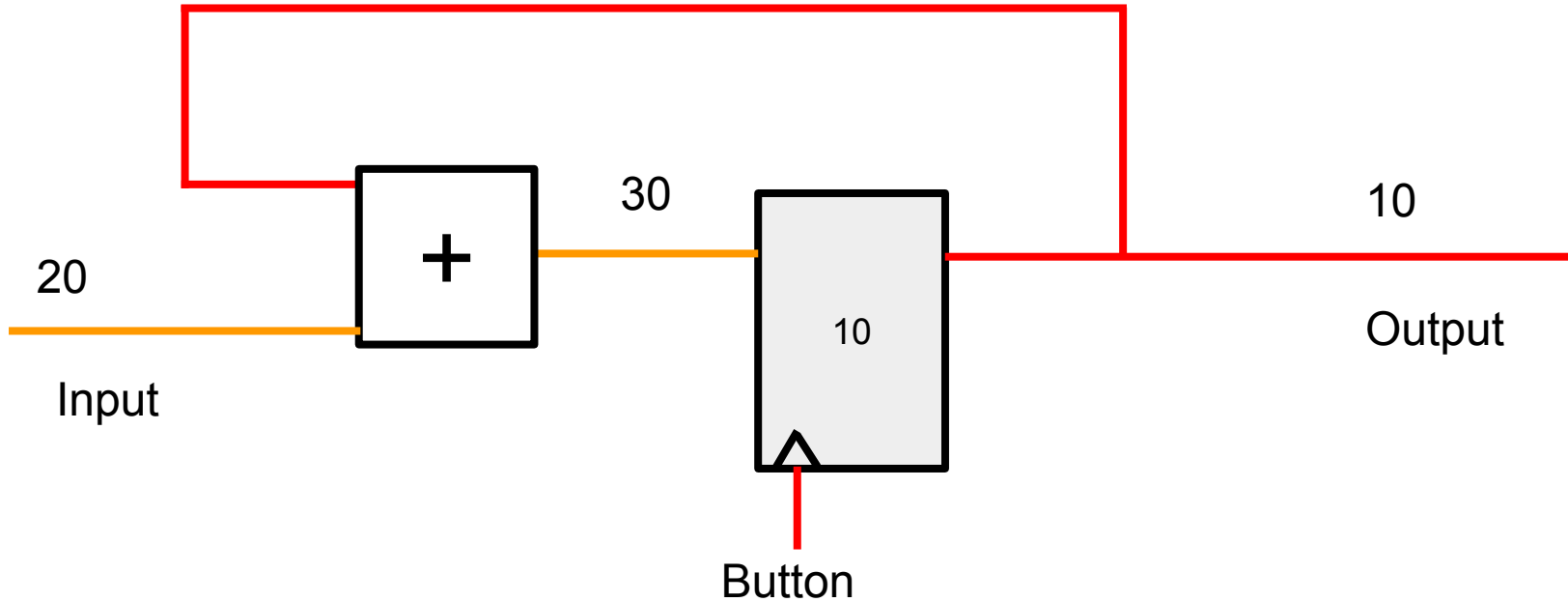
Example: Accumulator on input [10,20,30]



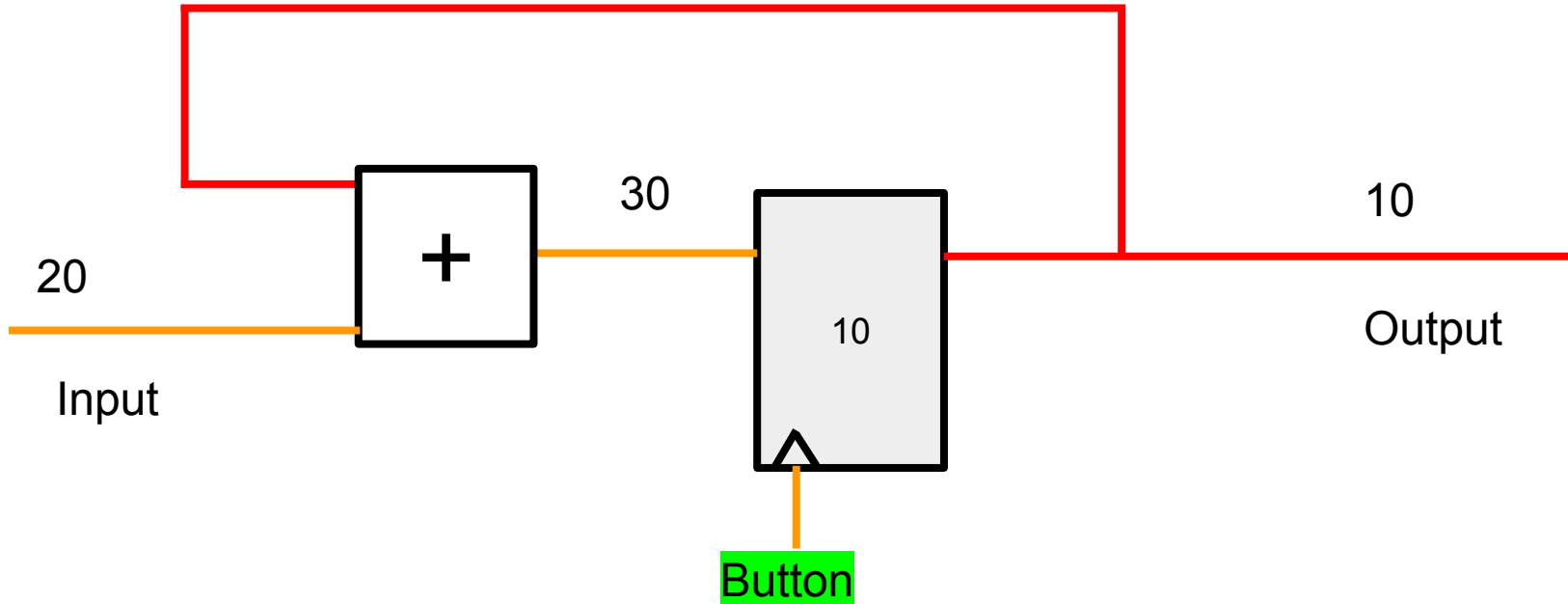
Example: Accumulator on input [10,20,30]



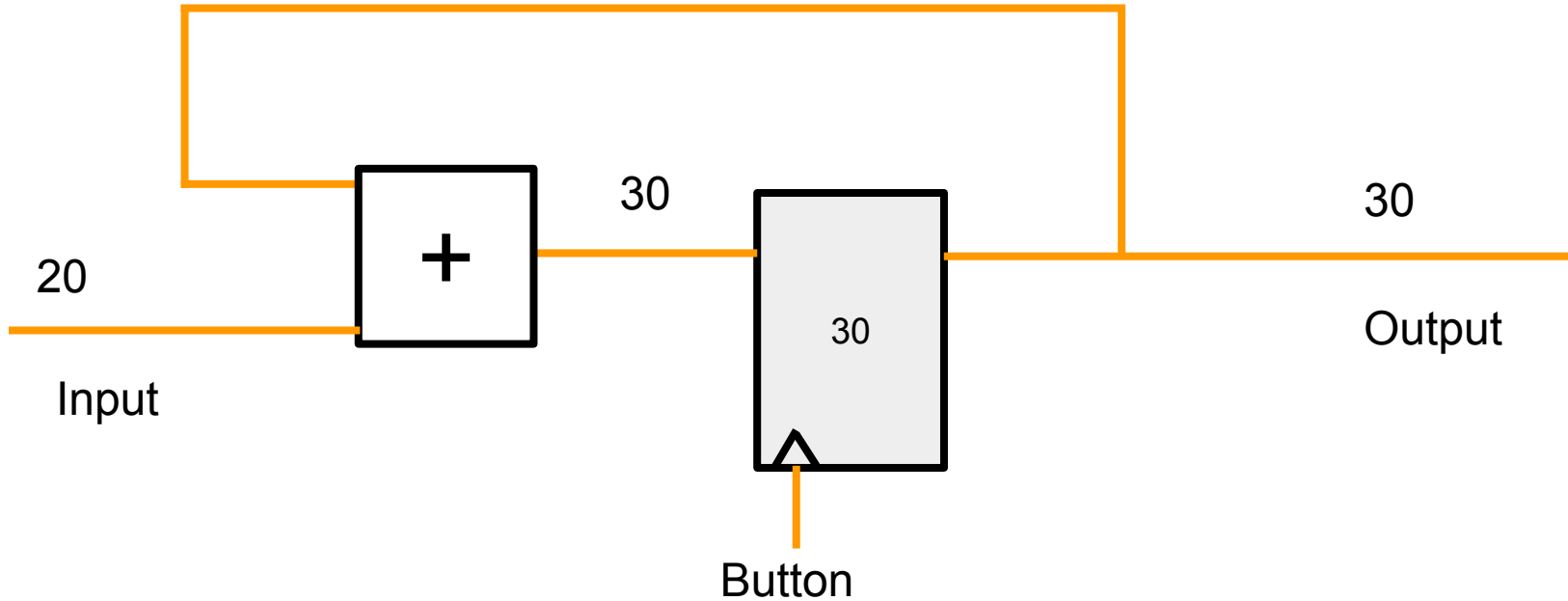
Example: Accumulator on input [10,20,30]



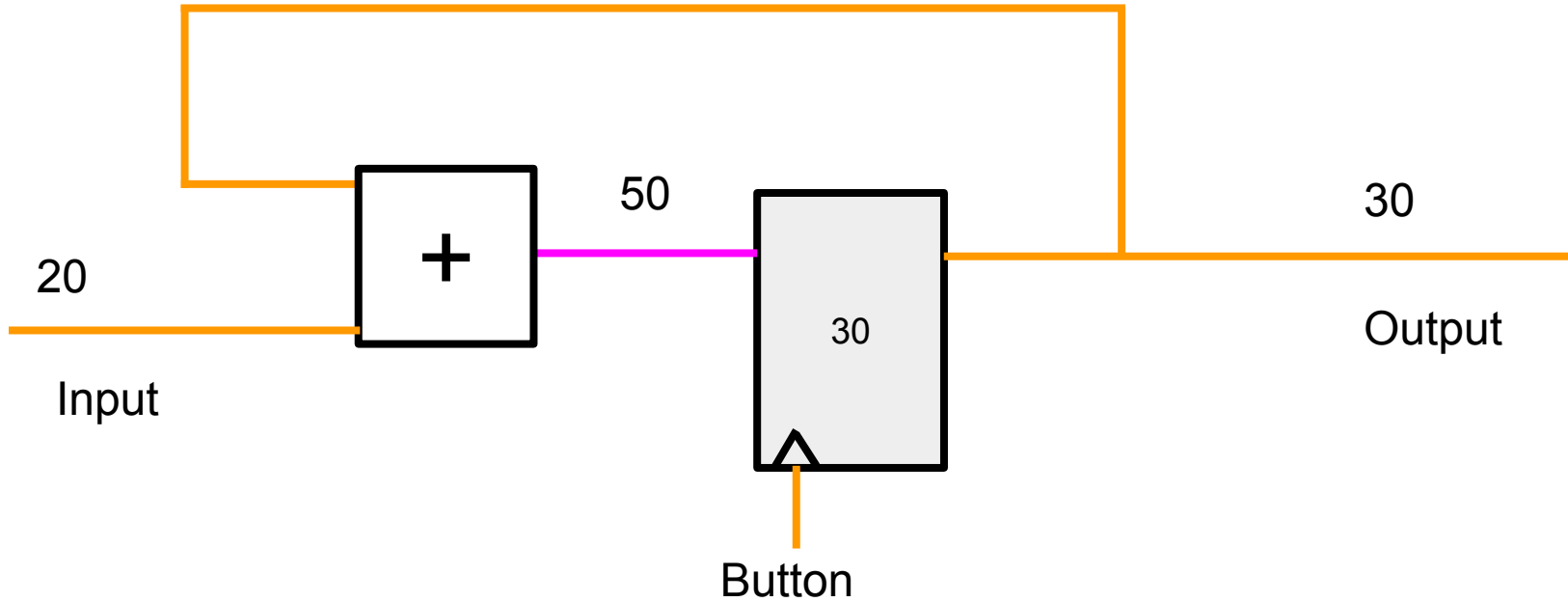
Example: Accumulator on input [10,20,30]



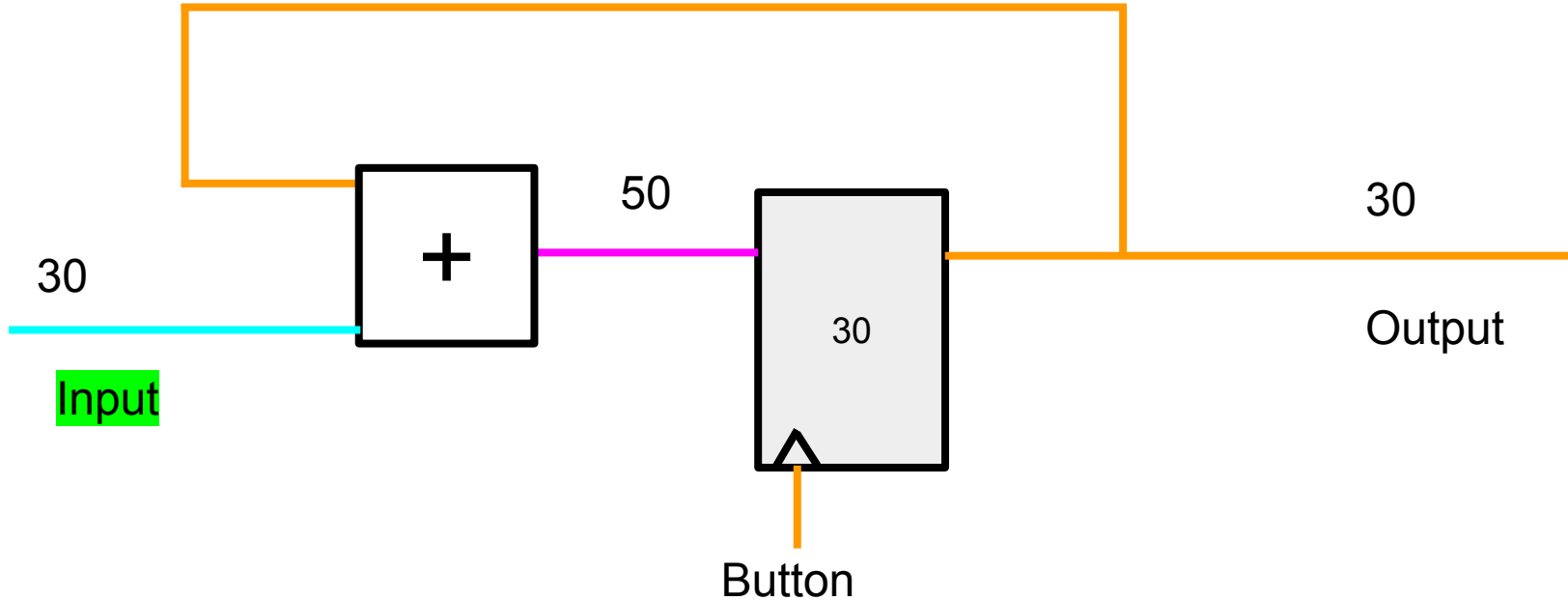
Example: Accumulator on input [10,20,30]



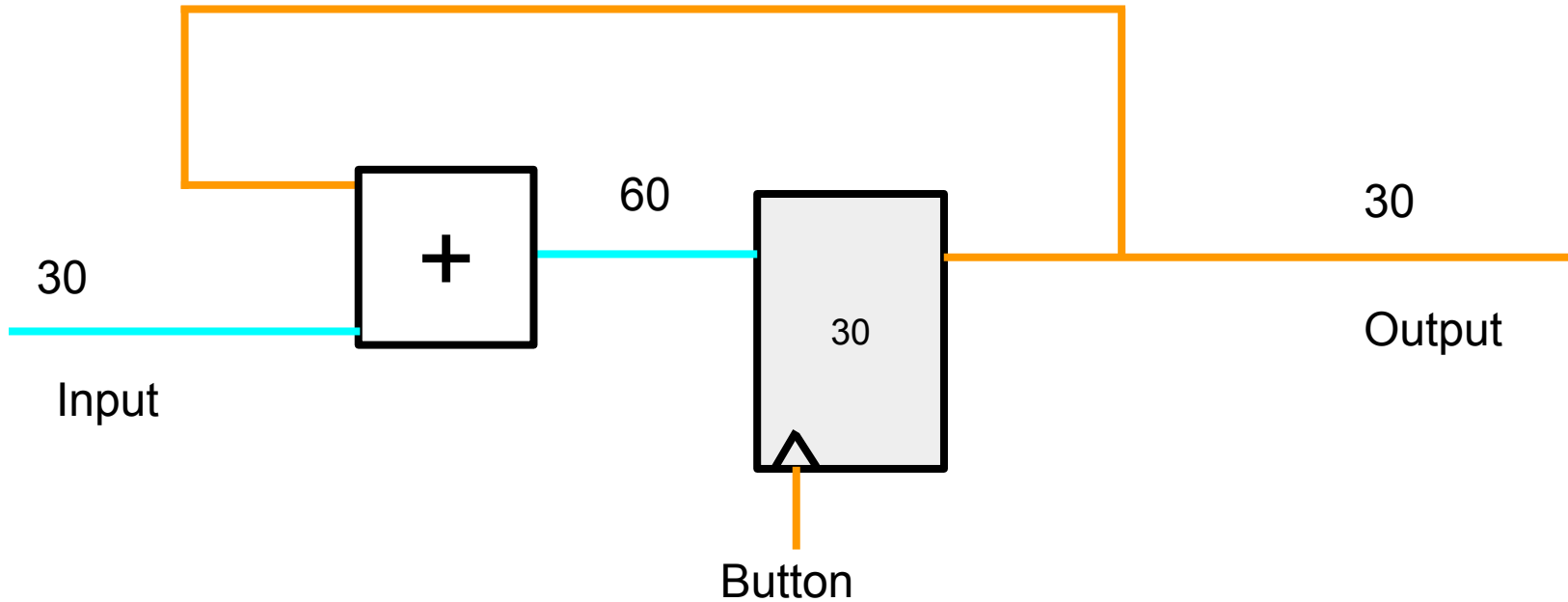
Example: Accumulator on input [10,20,30]



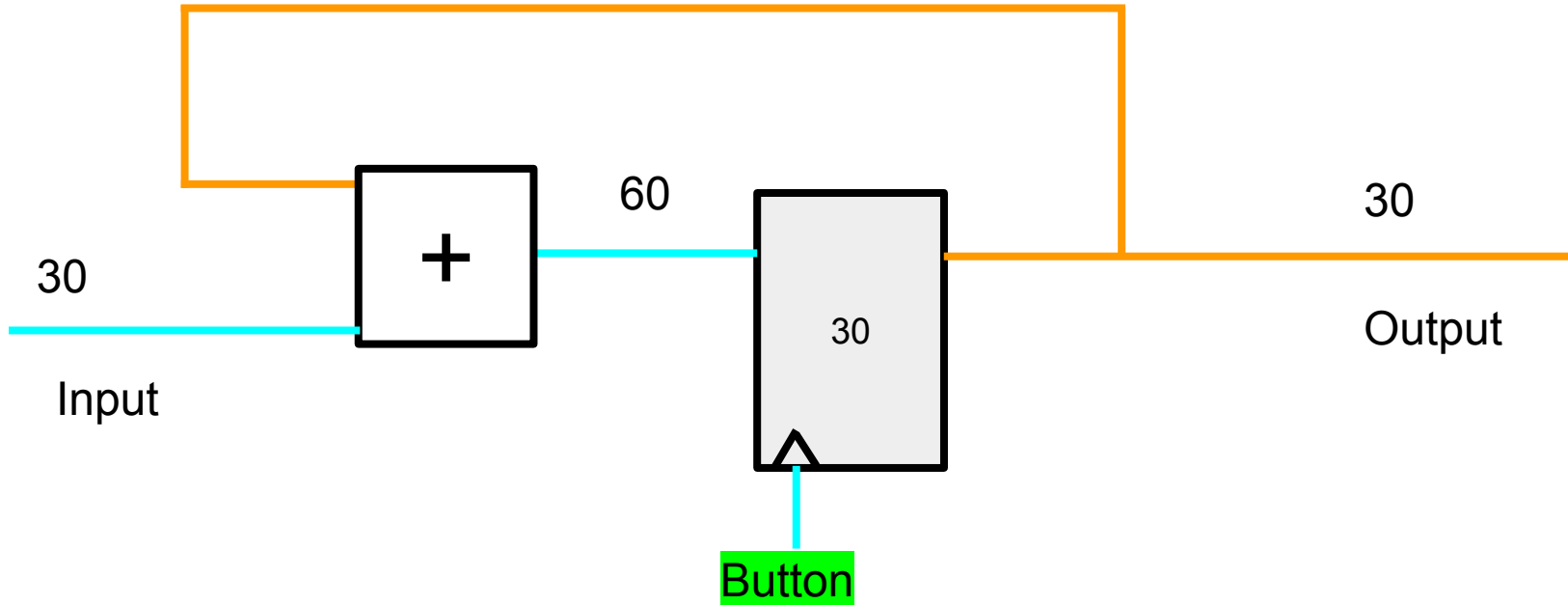
Example: Accumulator on input [10,20,30]



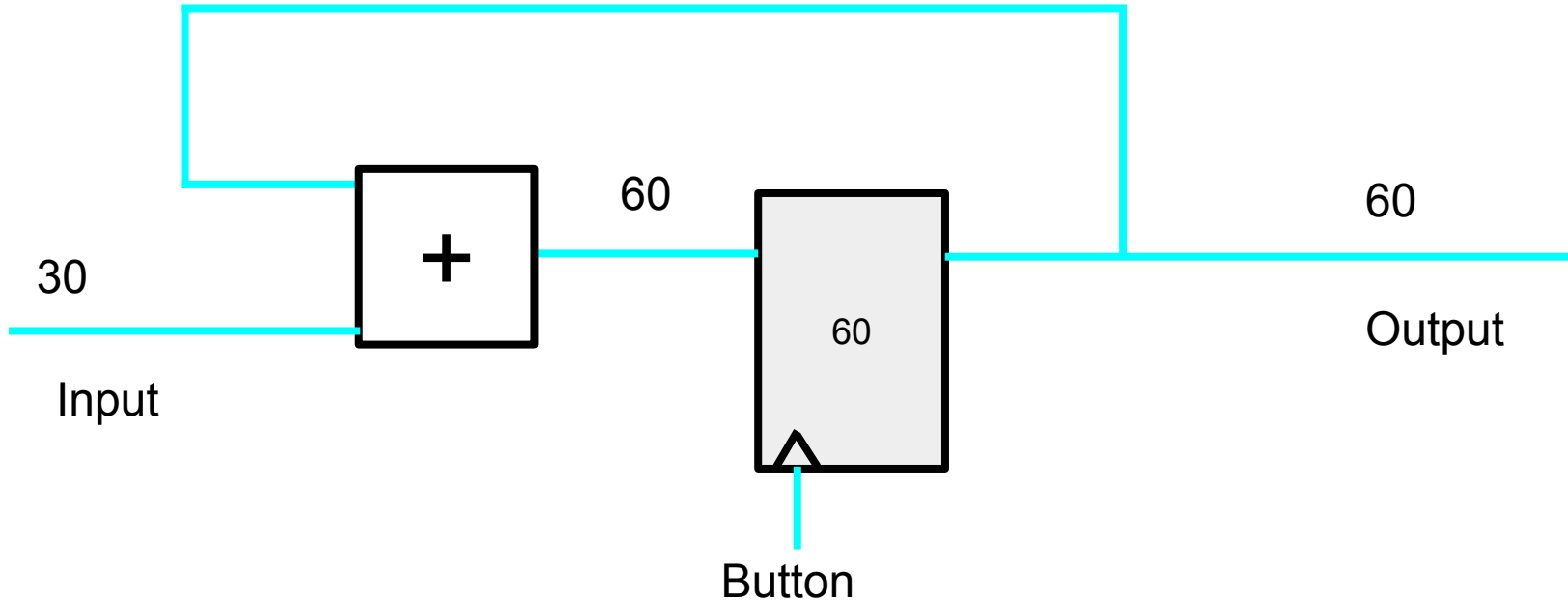
Example: Accumulator on input [10,20,30]



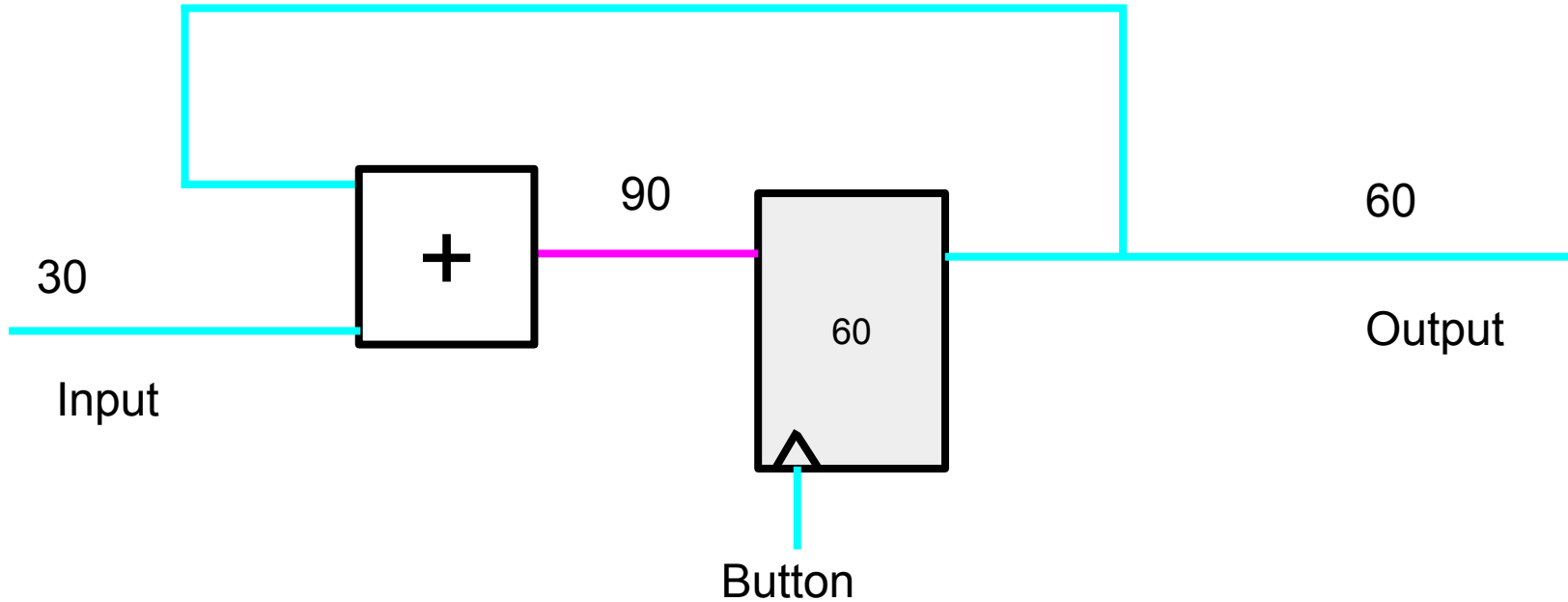
Example: Accumulator on input [10,20,30]



Example: Accumulator on input [10,20,30]



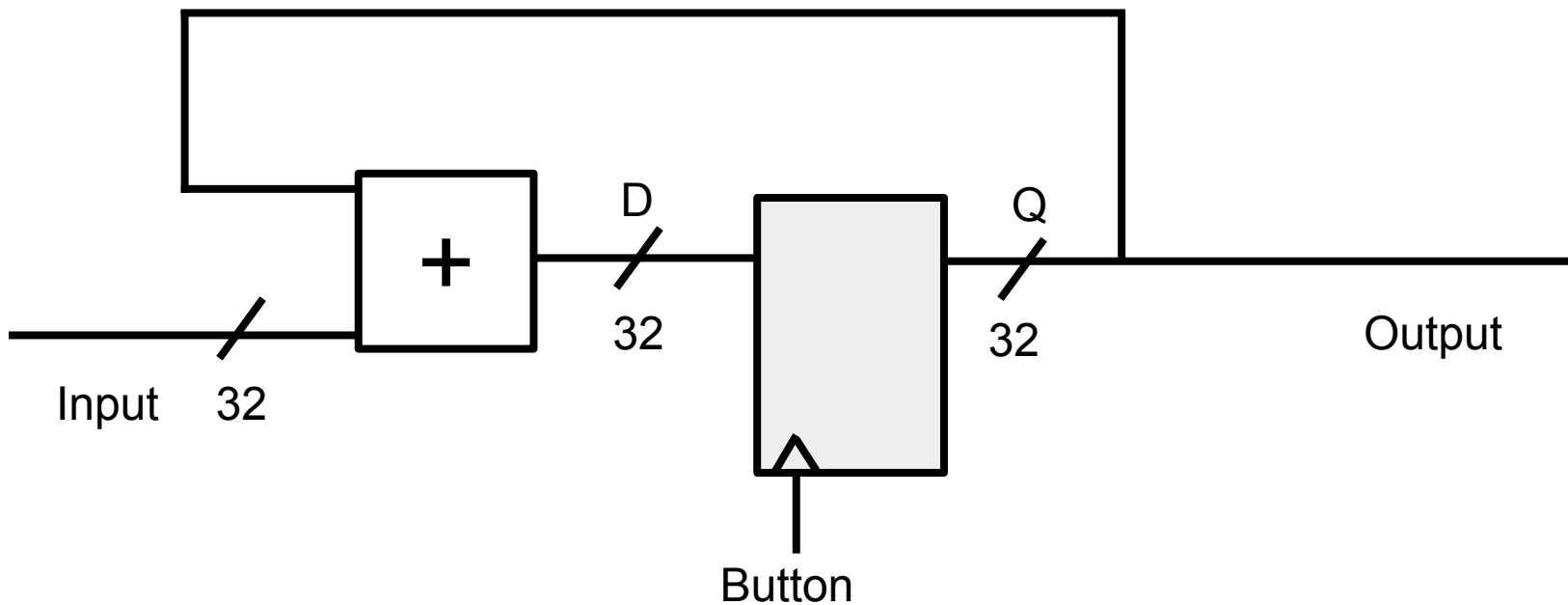
Example: Accumulator on input [10,20,30]



Agenda

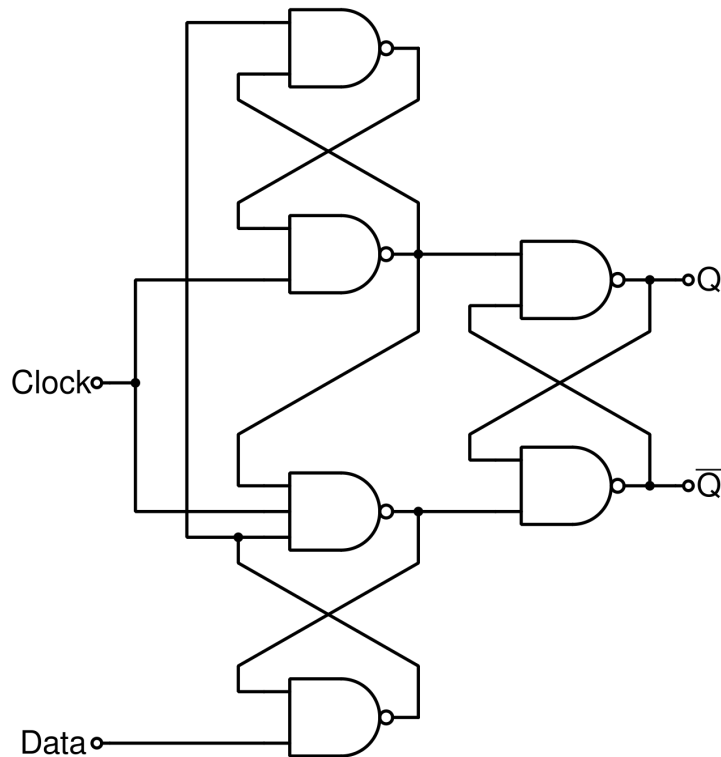
- Combinational Logic Delay
- Flip Flops and Registers
- **Register Timing**
- Clocks

How fast can I press the button?



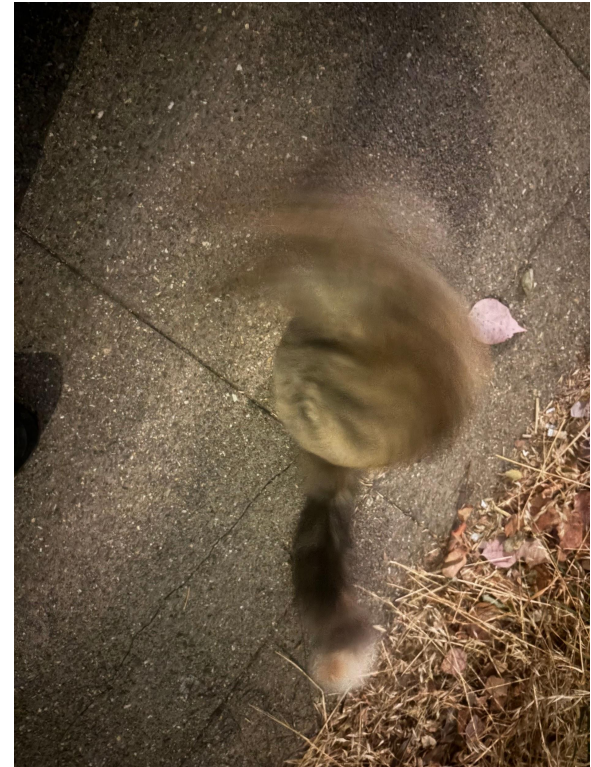
Register Timing

- The register is a circuit, which means it has a delay too...
- In addition, the feedback loops of the latches require that **data be stable** for some time before and after the clock trigger



Analogy: Blurry Photos

- Imagine we try to take a picture of something.
- If the subject is moving as we take a picture, the result can be blurry
- In a register, this means we don't capture data properly and the register is put in **an undefined state**



Analogy: Blurry Photos

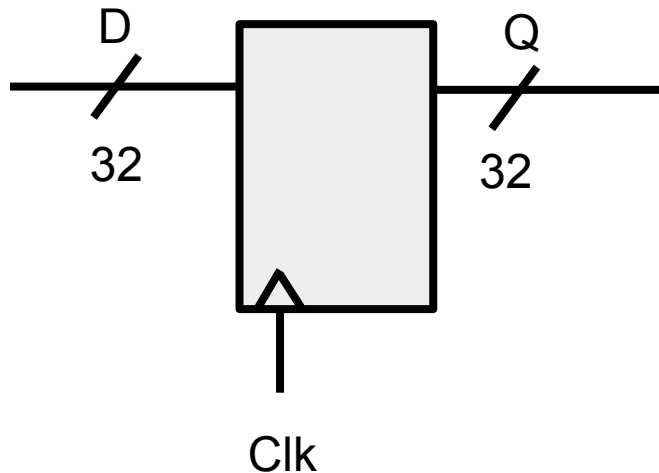
- To prevent this, we need the input to be stable for some time **before the clock trigger**, and some time **after the clock trigger**.



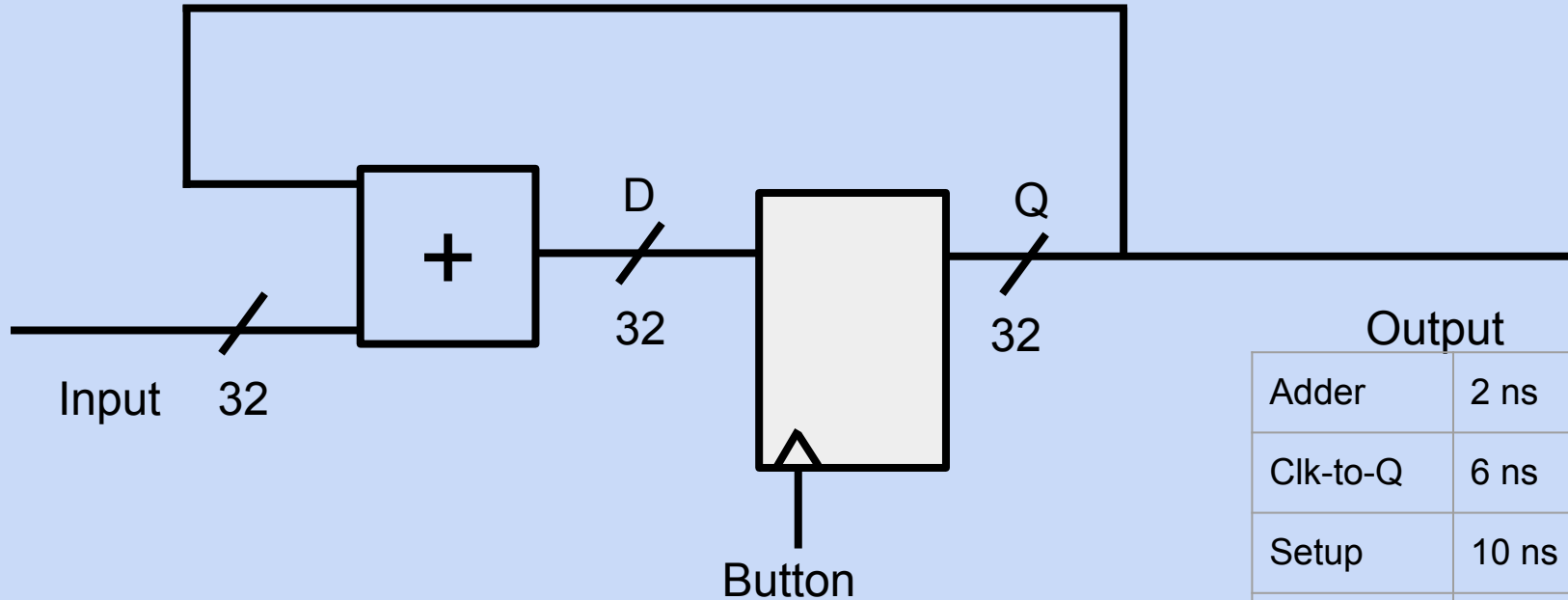
Register Timings

For this class, three relevant values:

- Clk-to-Q: delay between clock and when Q gets updated (delay of the register)
- Setup time: Time D needs to be stable **before** a clock trigger
- Hold time: Time D needs to be stable **after** a clock trigger



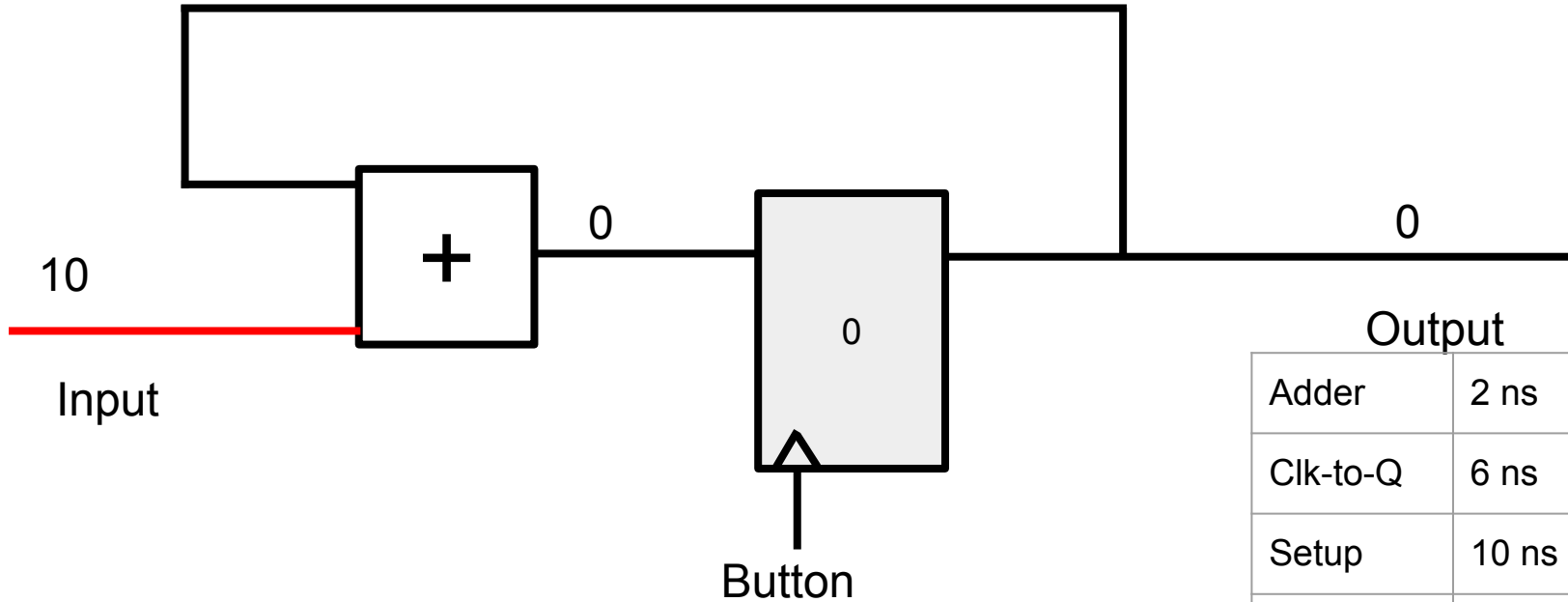
How fast can I press the button?



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

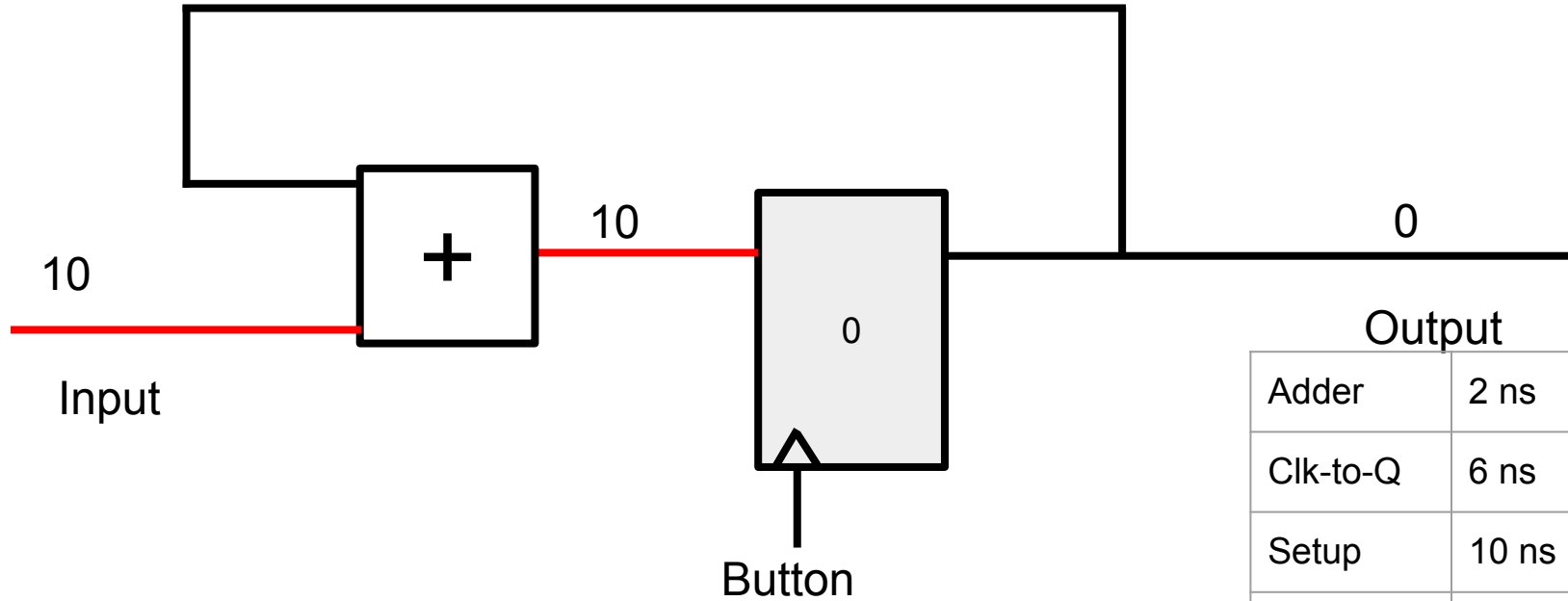
Time 0



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

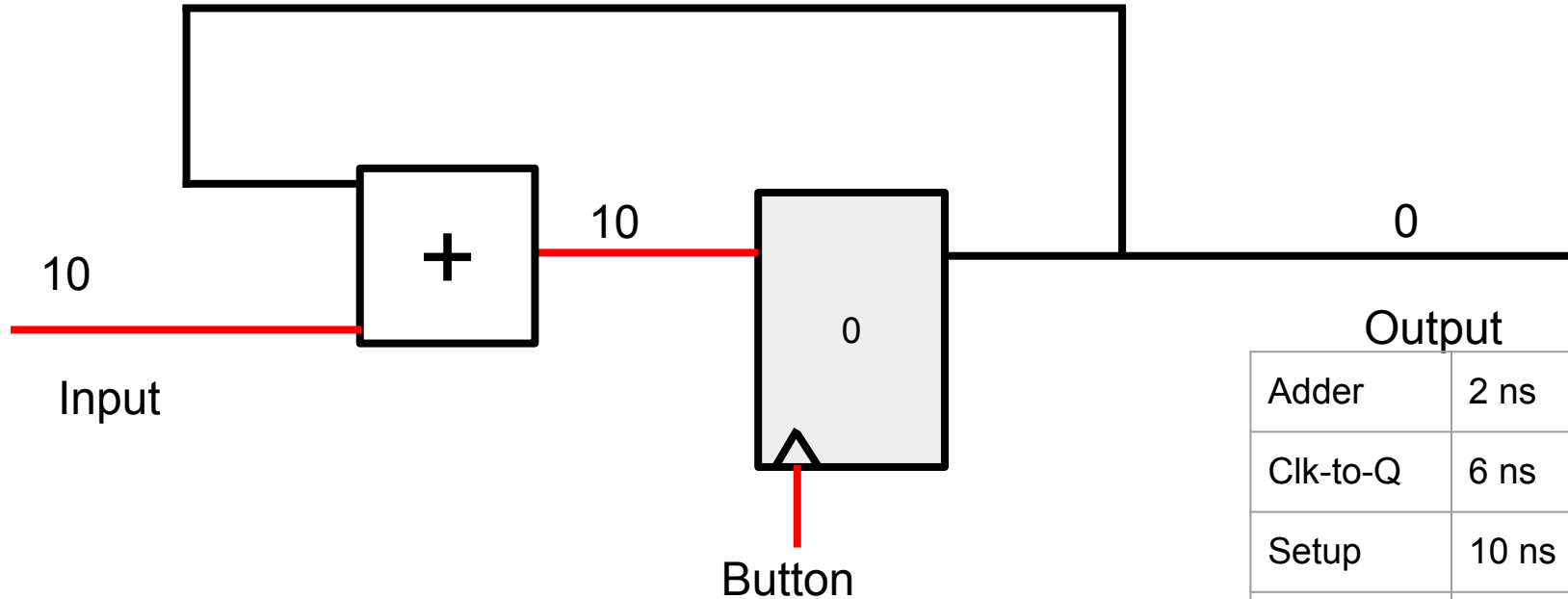
Time 2



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

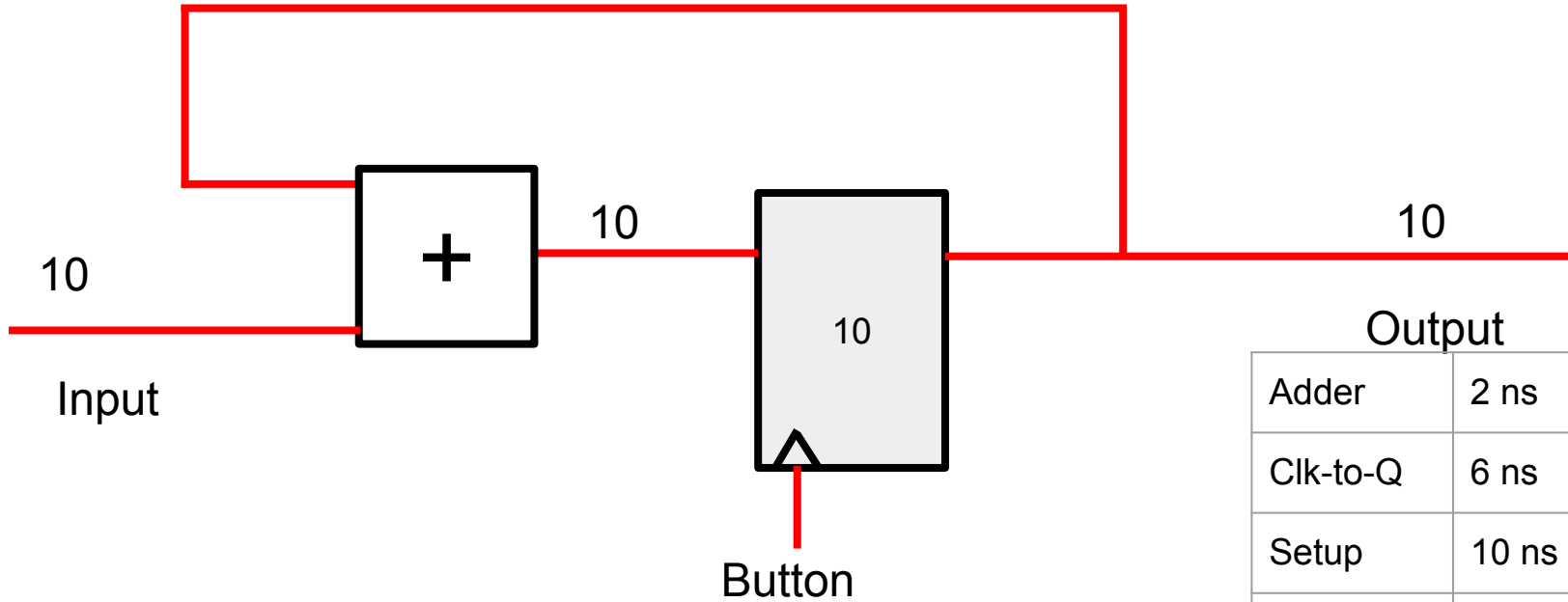
Time 12: Earliest I can press the button (setup time)



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

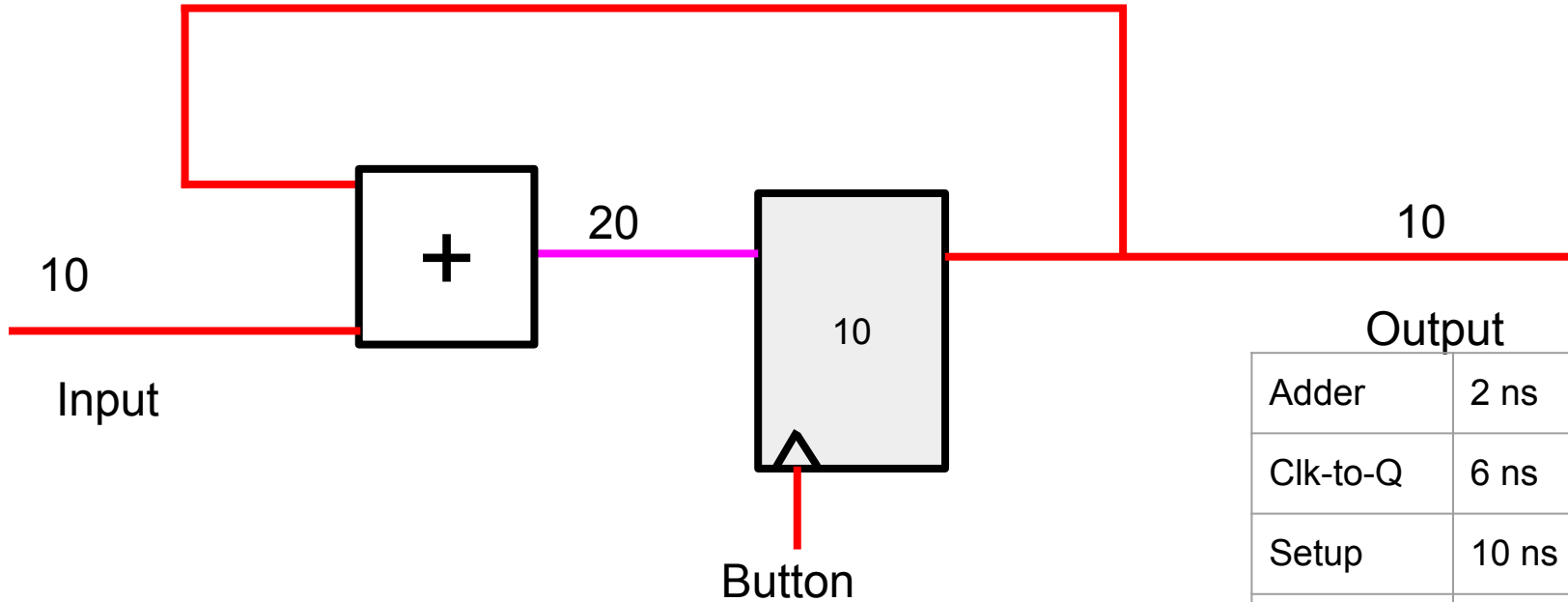
Time 18:



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

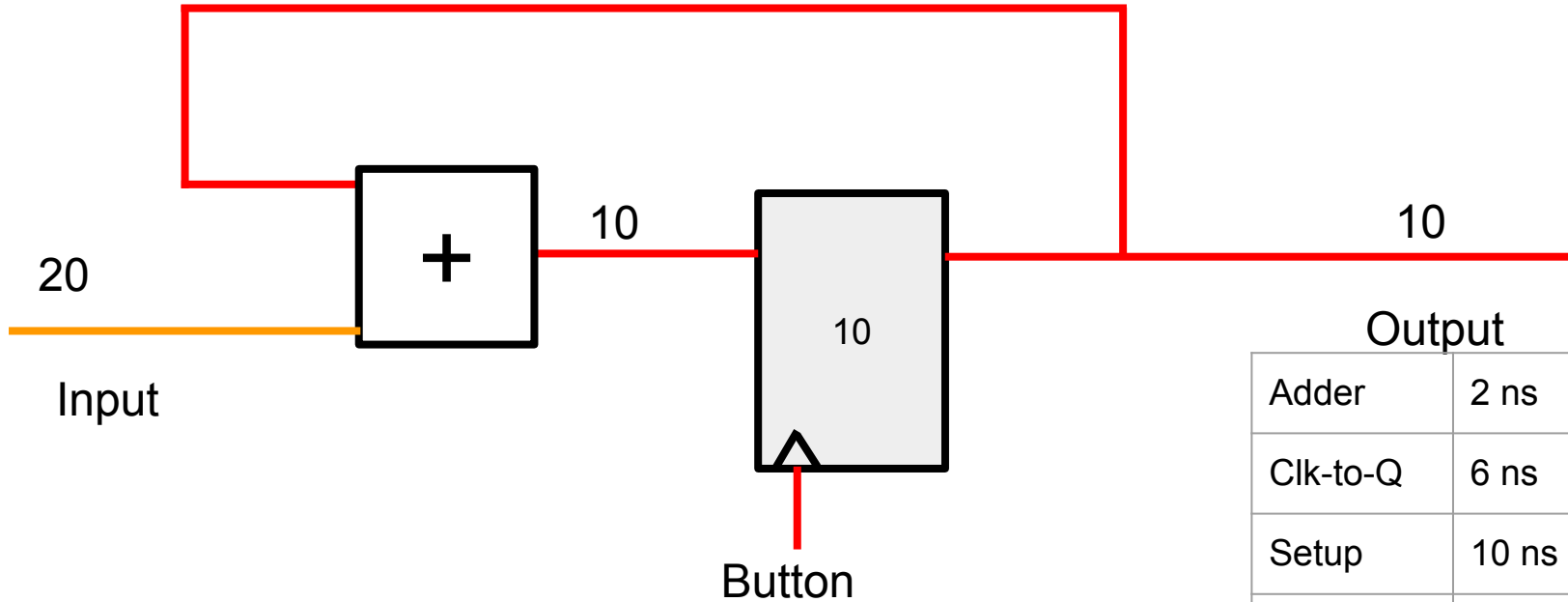
Time 20... but when should I update input?



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

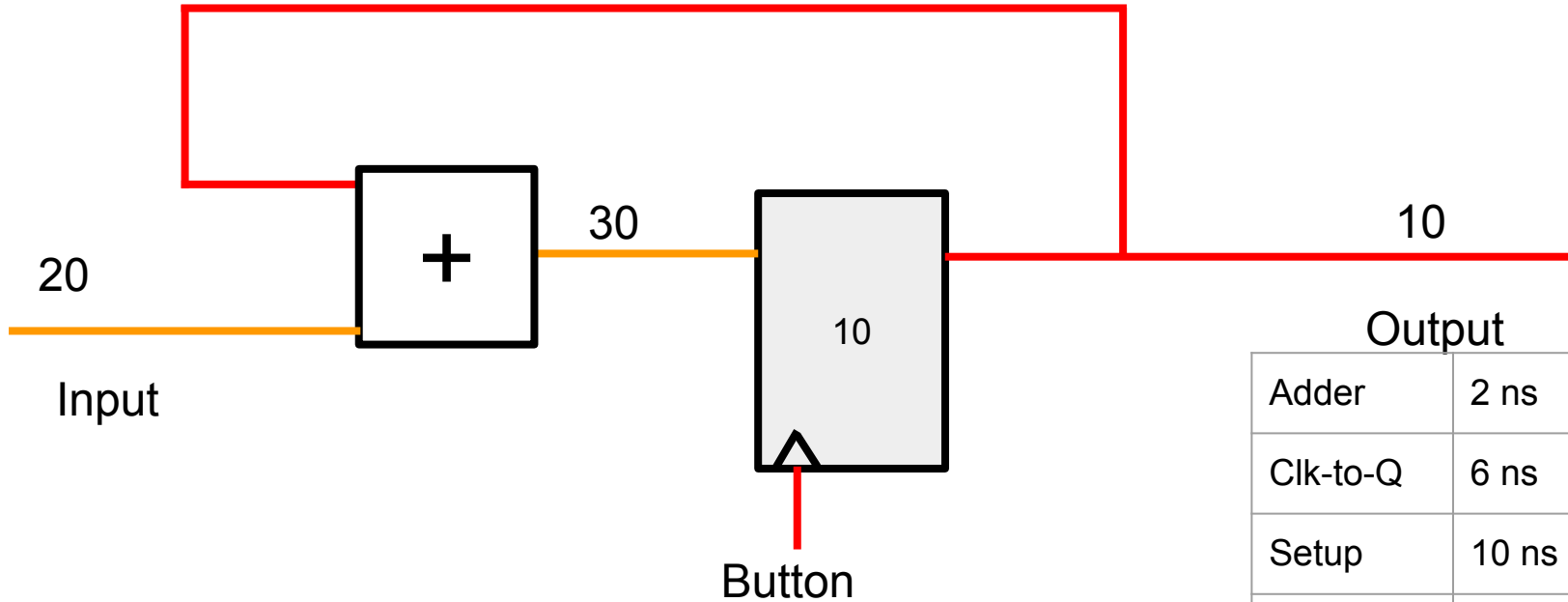
Latest update: Time 18



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

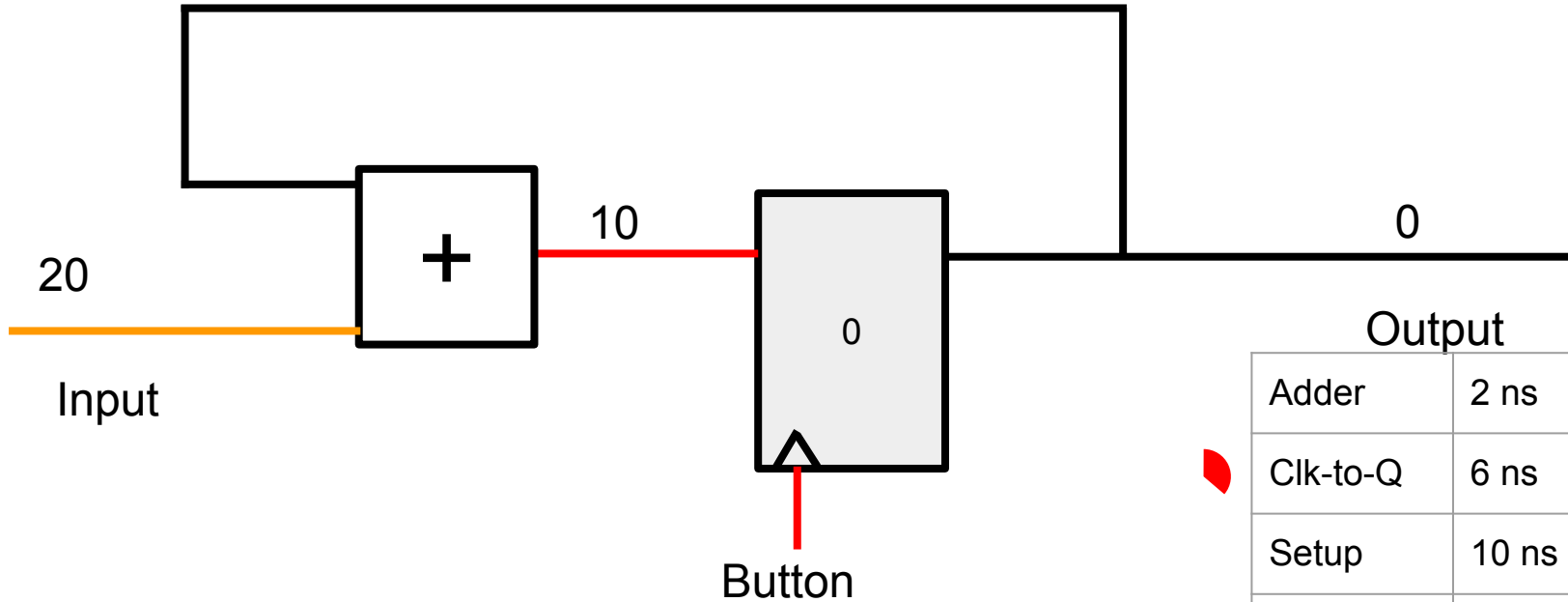
Latest update: Time 20



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

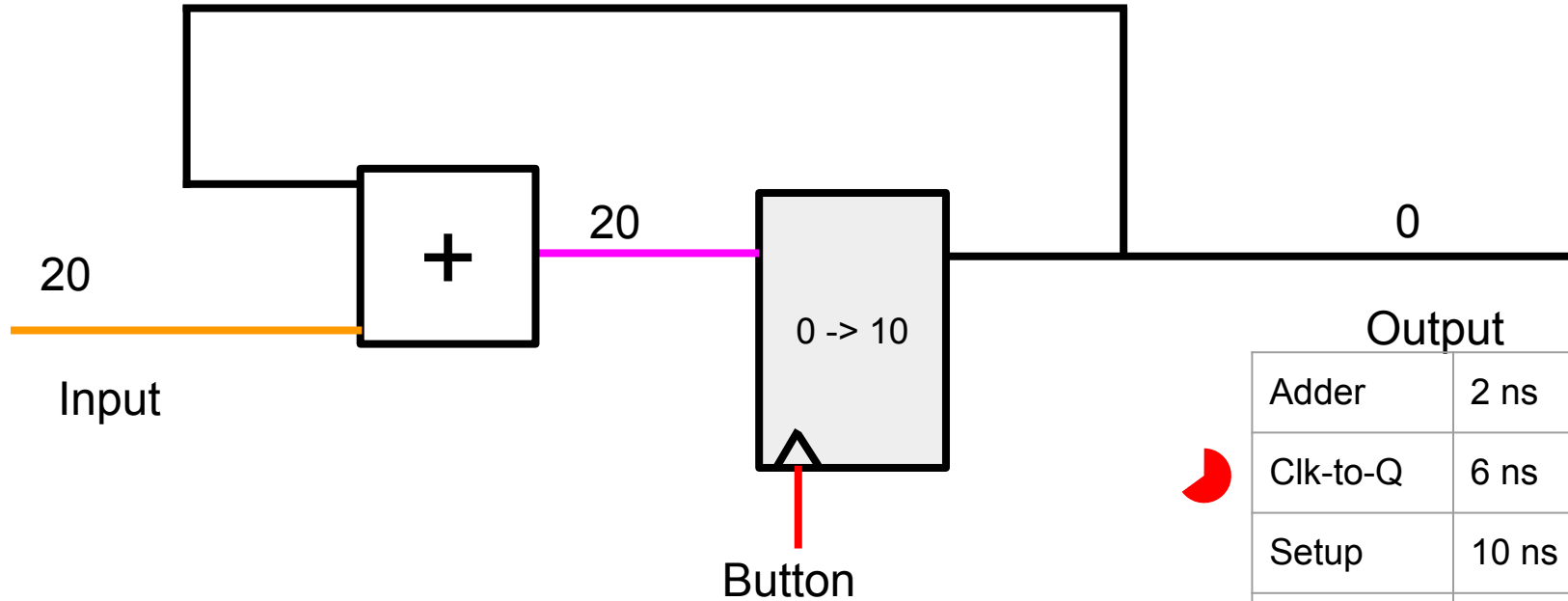
Earliest update: Time 14



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

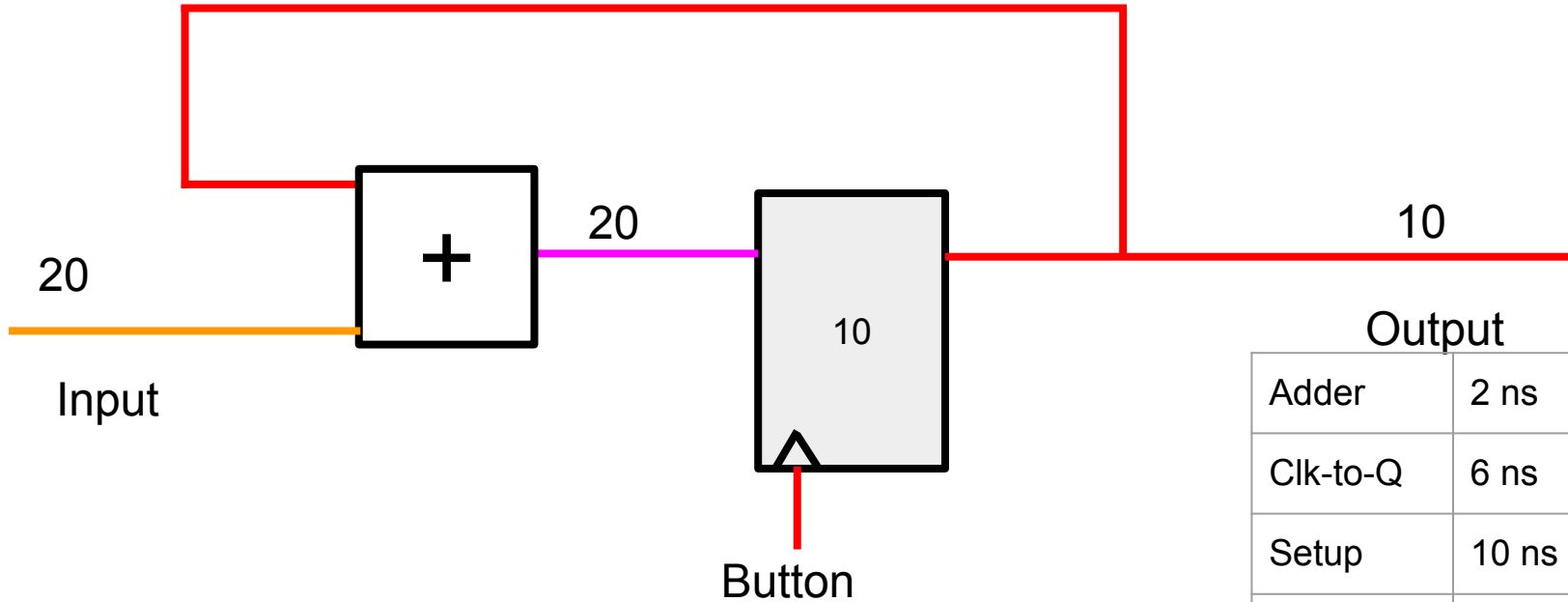
Earliest update: Time 16



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

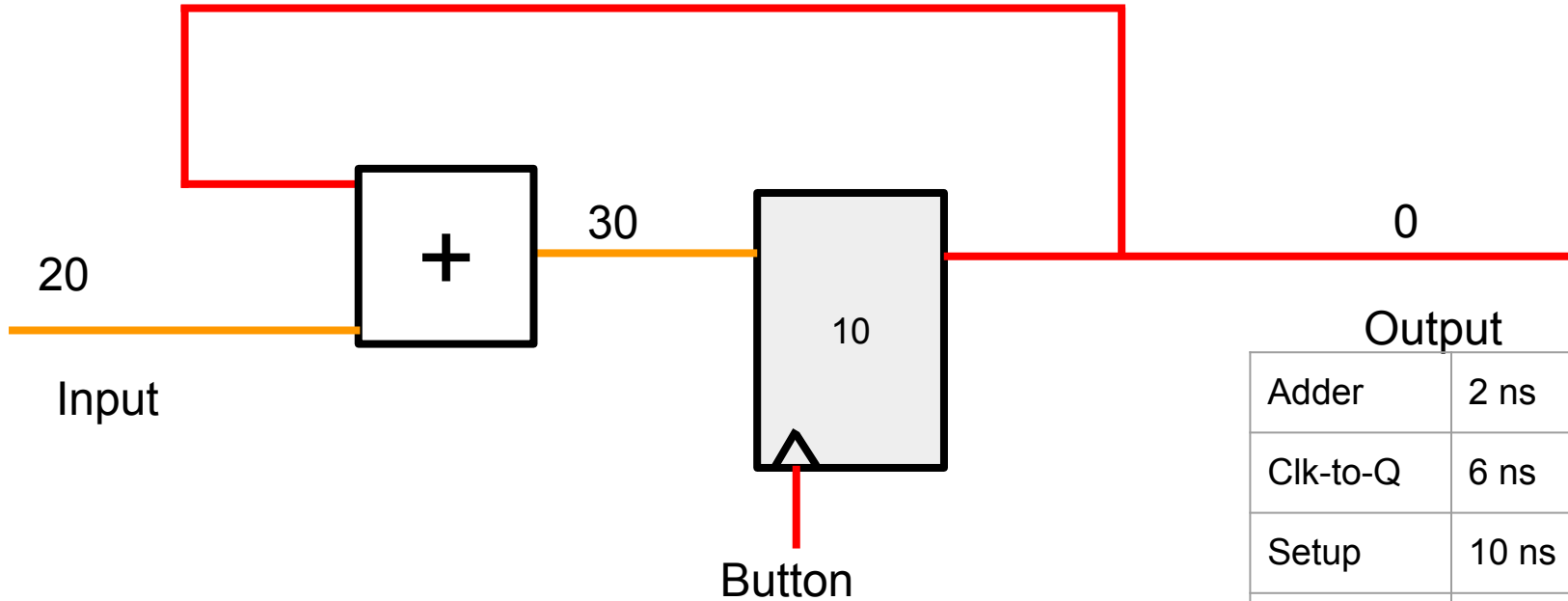
Earliest update: Time 18



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

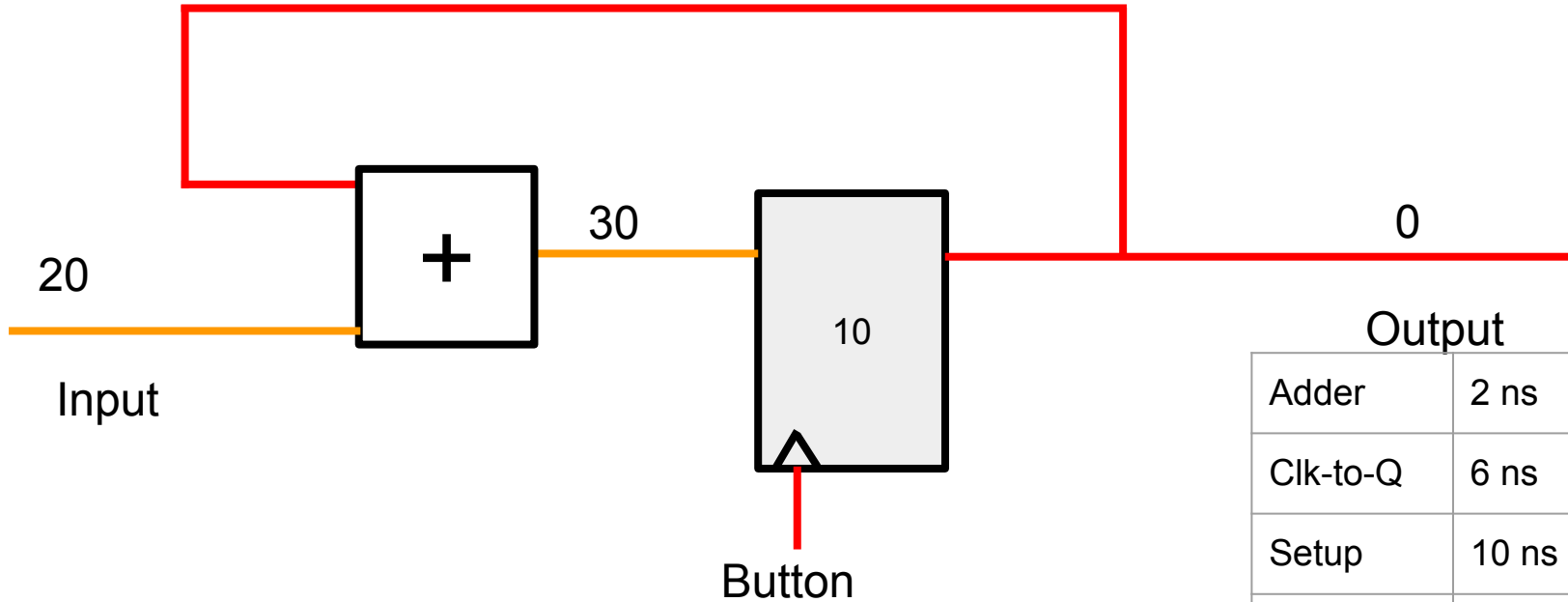
Earliest update: Time 20



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

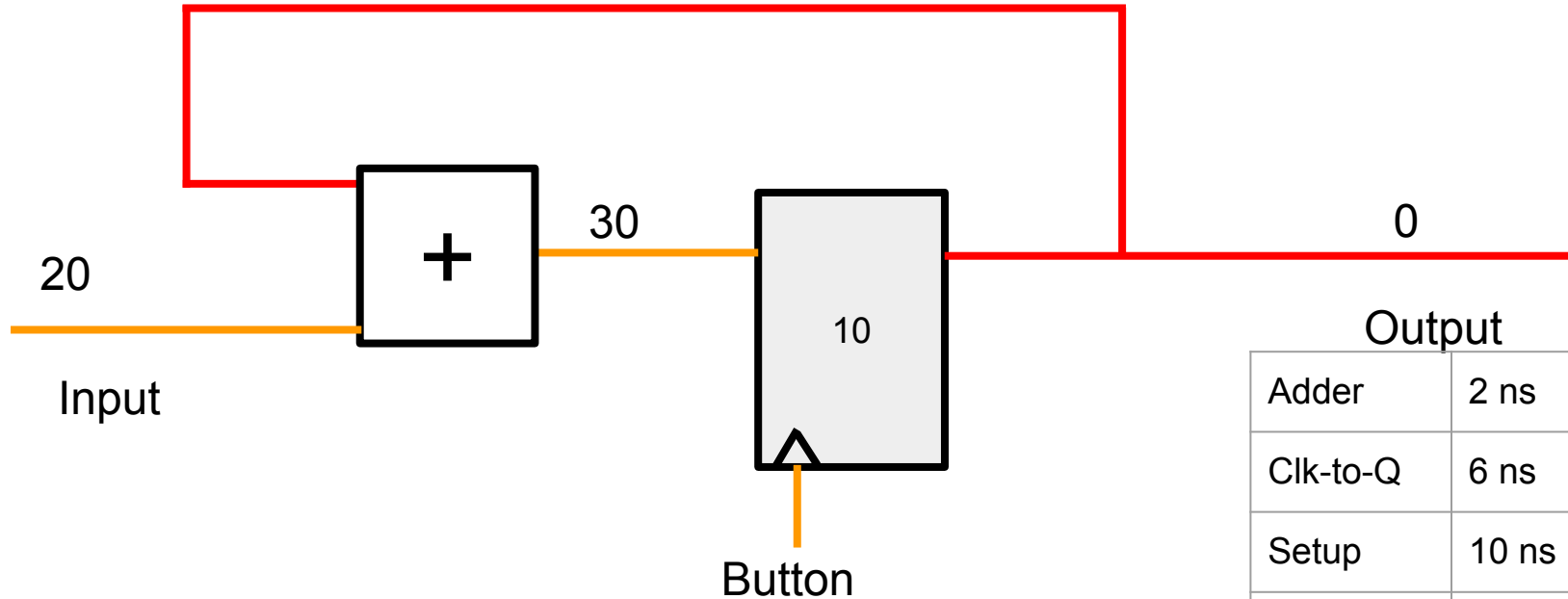
Time 20: Register D becomes stable



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

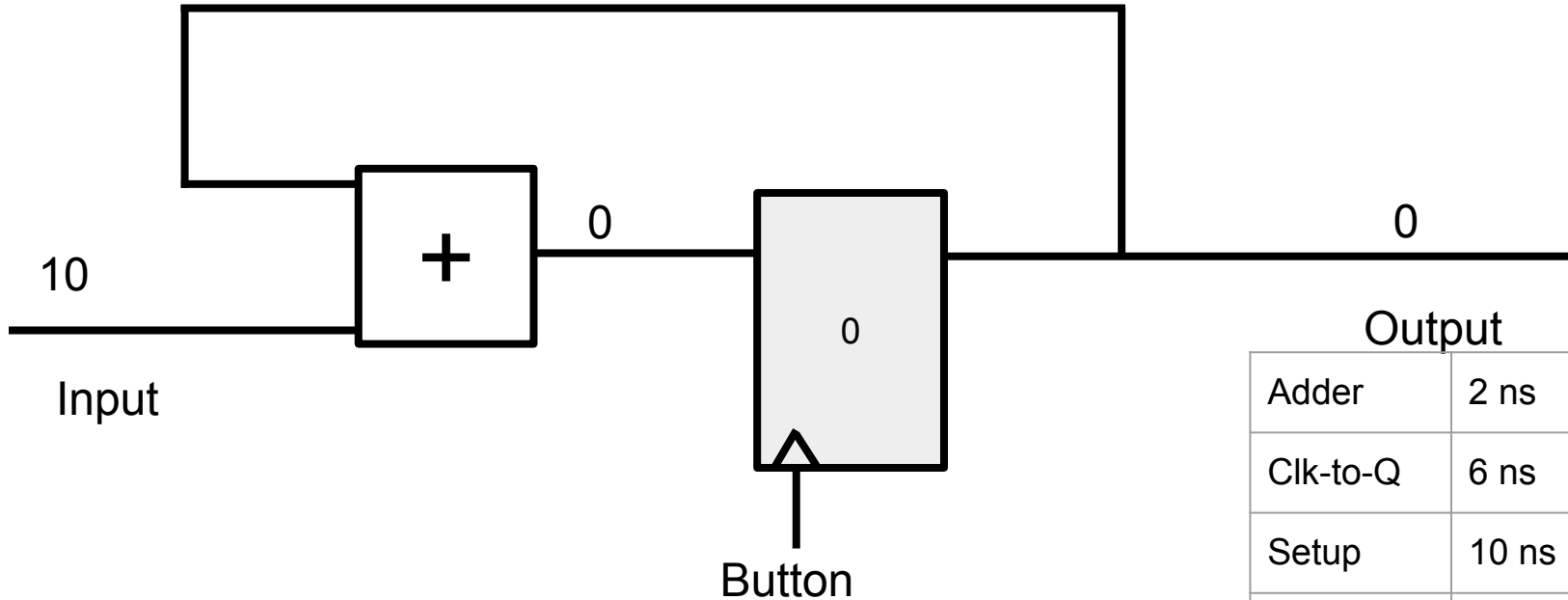
Time 30: Setup time resolves, next button press possible



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

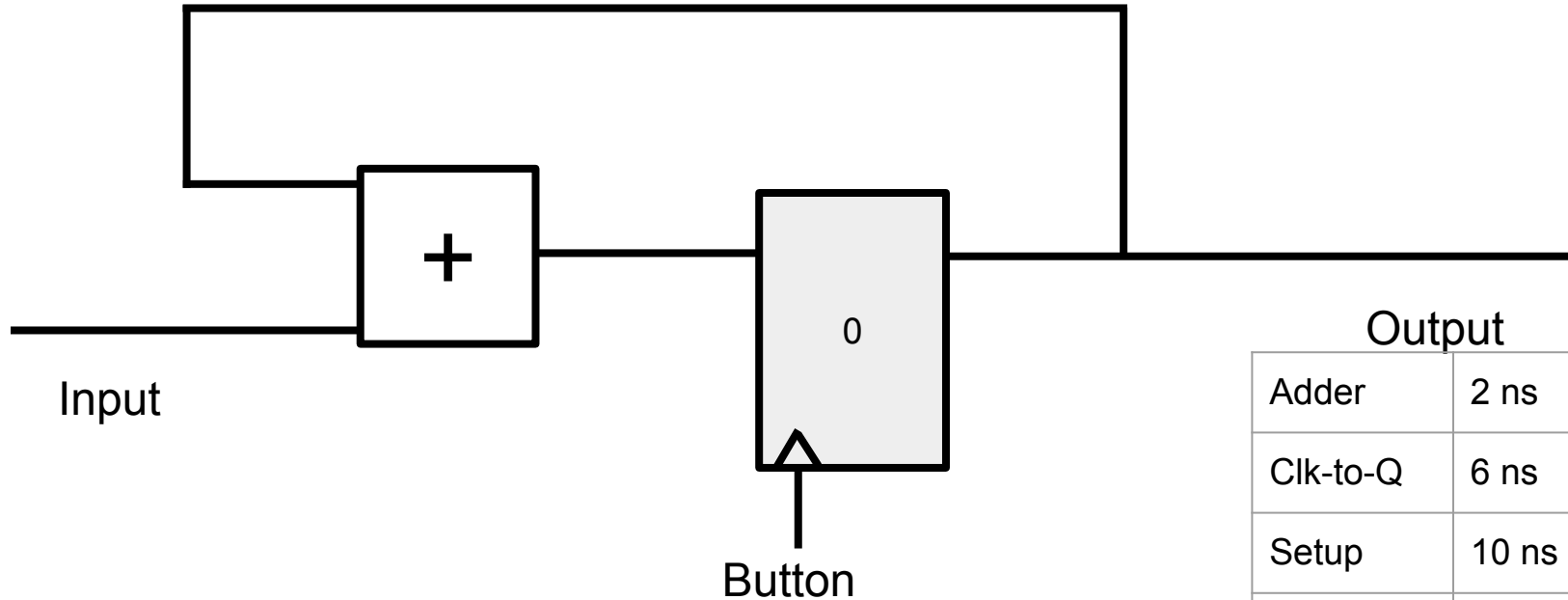
Button press at 12 and 30 -> 18 ns between presses



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

Input can arrive +2 ns to +6 ns from clk for optimal pace



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

Agenda

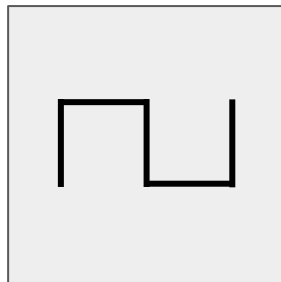
- Combinational Logic Delay
- Flip Flops and Registers
- Register Timing
- **Clocks**

Clock

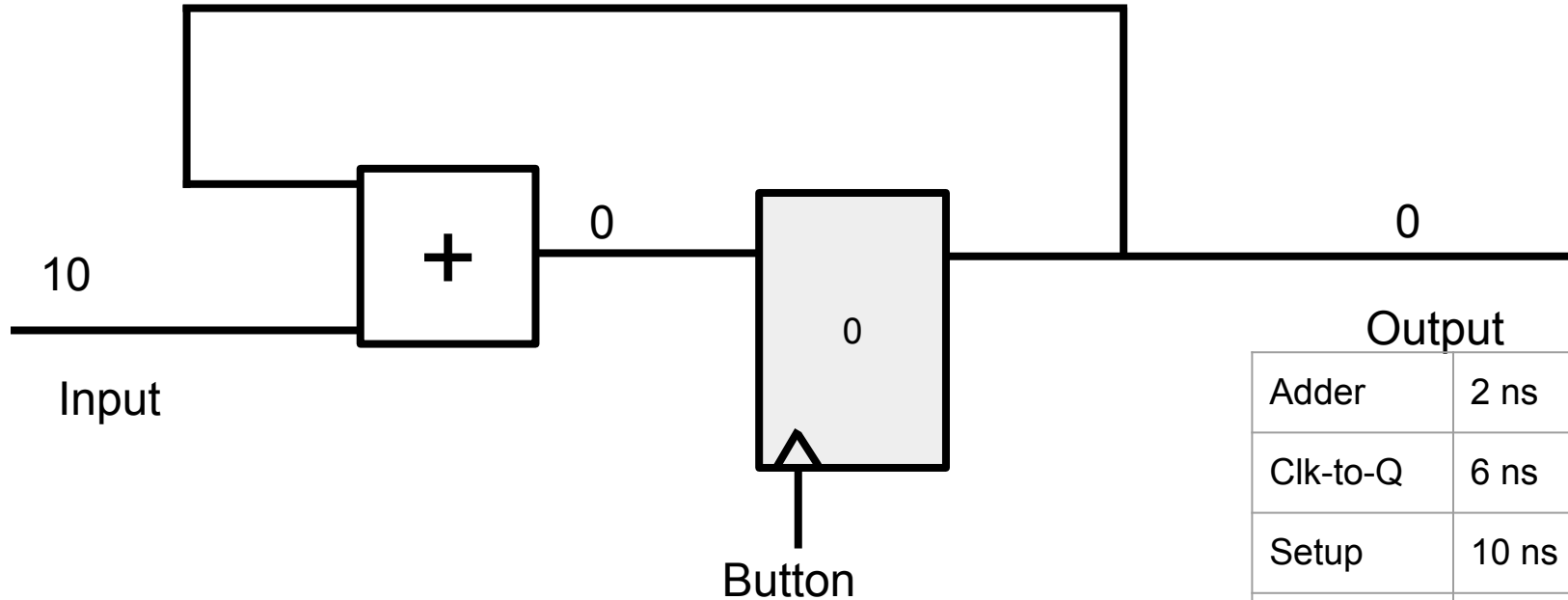
Rather than manually pressing a button to trigger clock cycles, we'll use a **clock** component.

- Made with a vibrating crystal to generate a rising edge every X ns.

Often we ask what the **minimum clock period** is for a circuit.



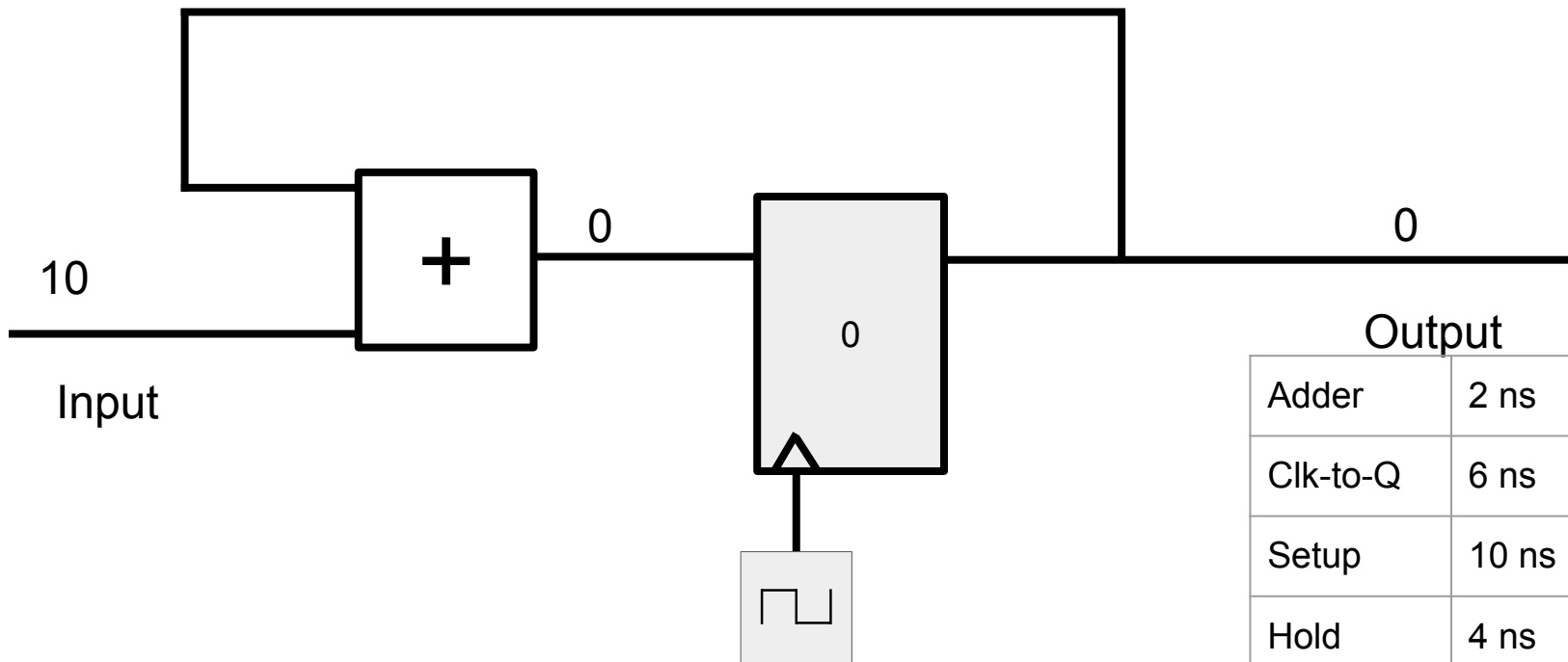
Button press at 12 and 30 -> 18 ns between presses



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

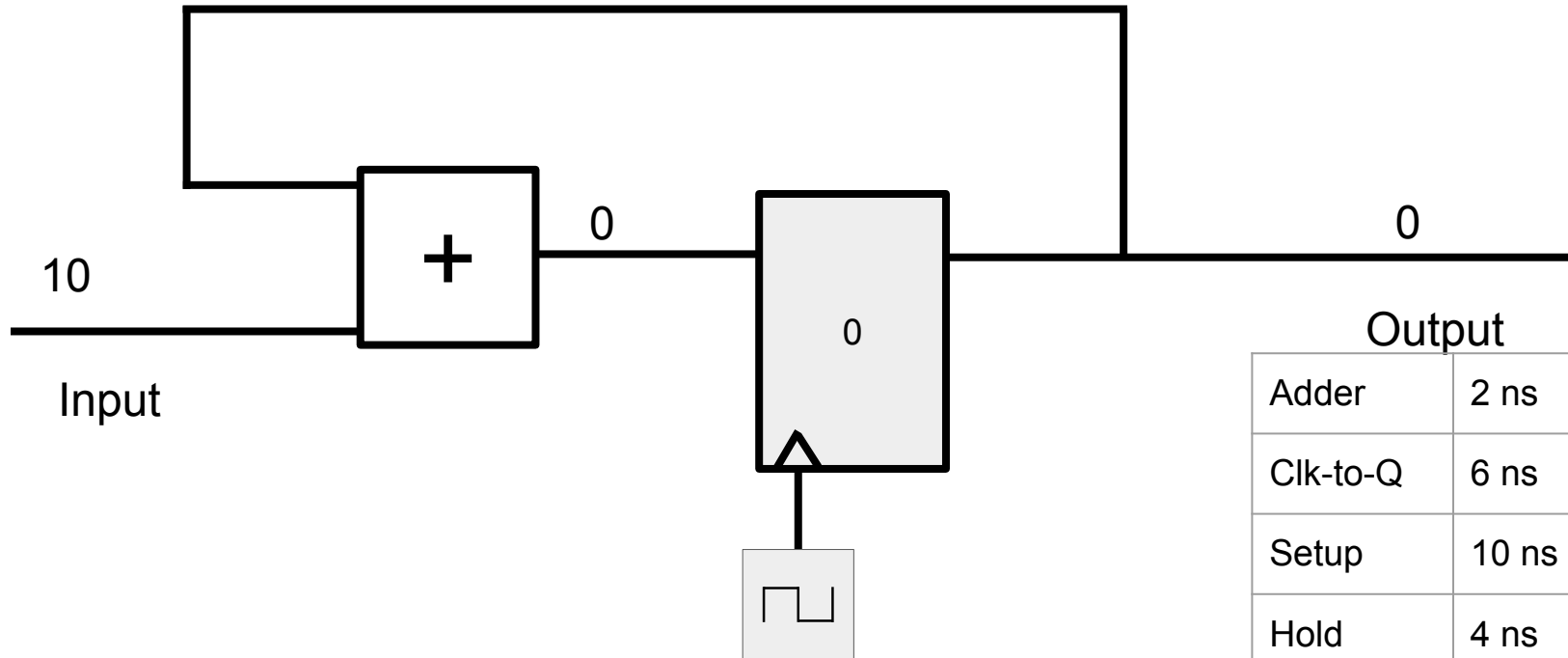
Minimum clock period: 18 ns. How to compute in general?



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

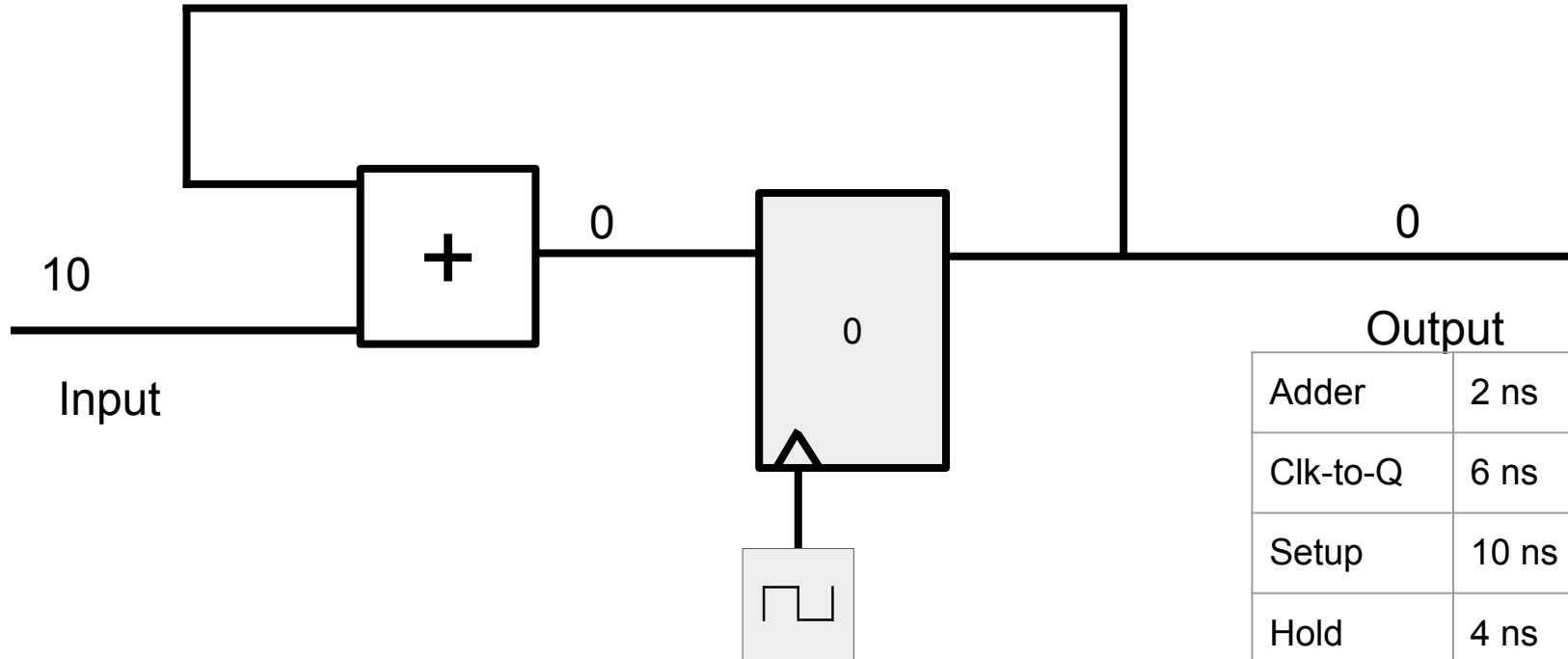
Min clock period = Clk-to-Q + Max logic delay + Setup



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

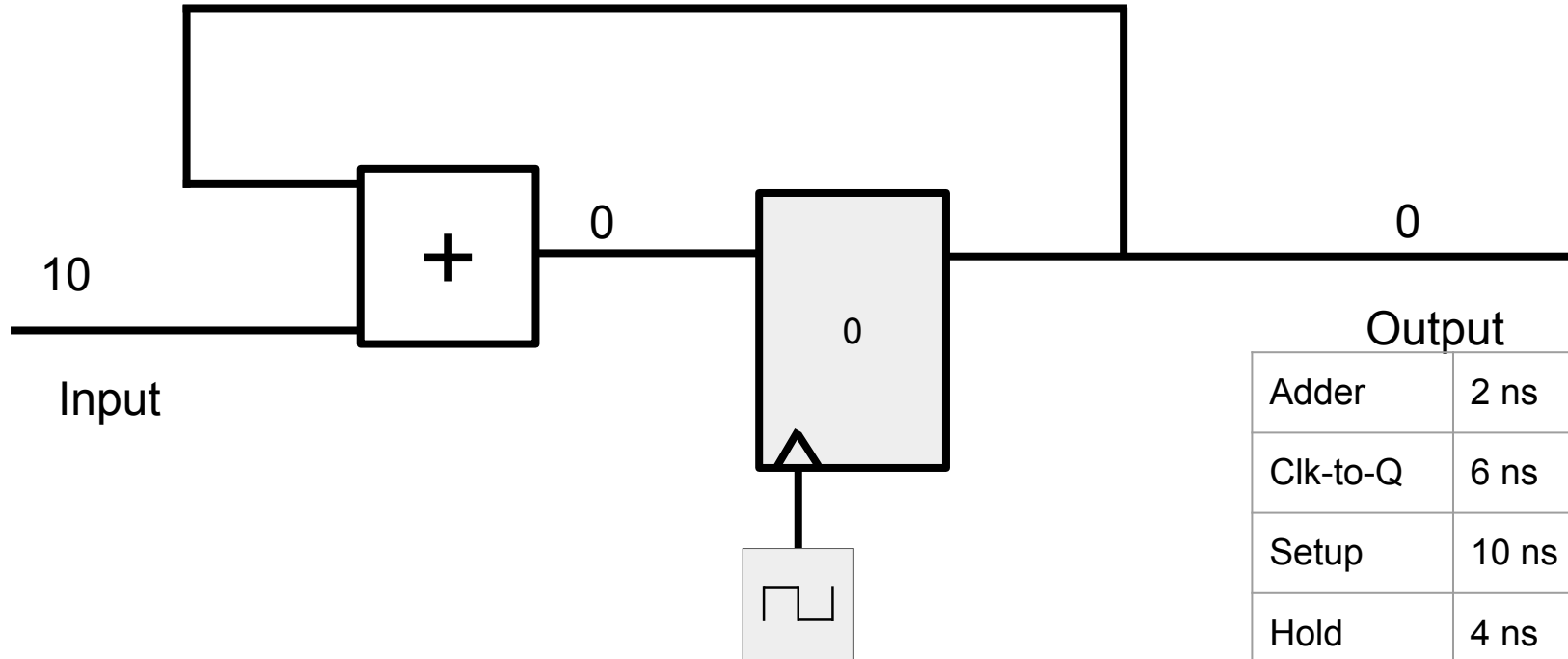
How to prevent hold time violations?



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

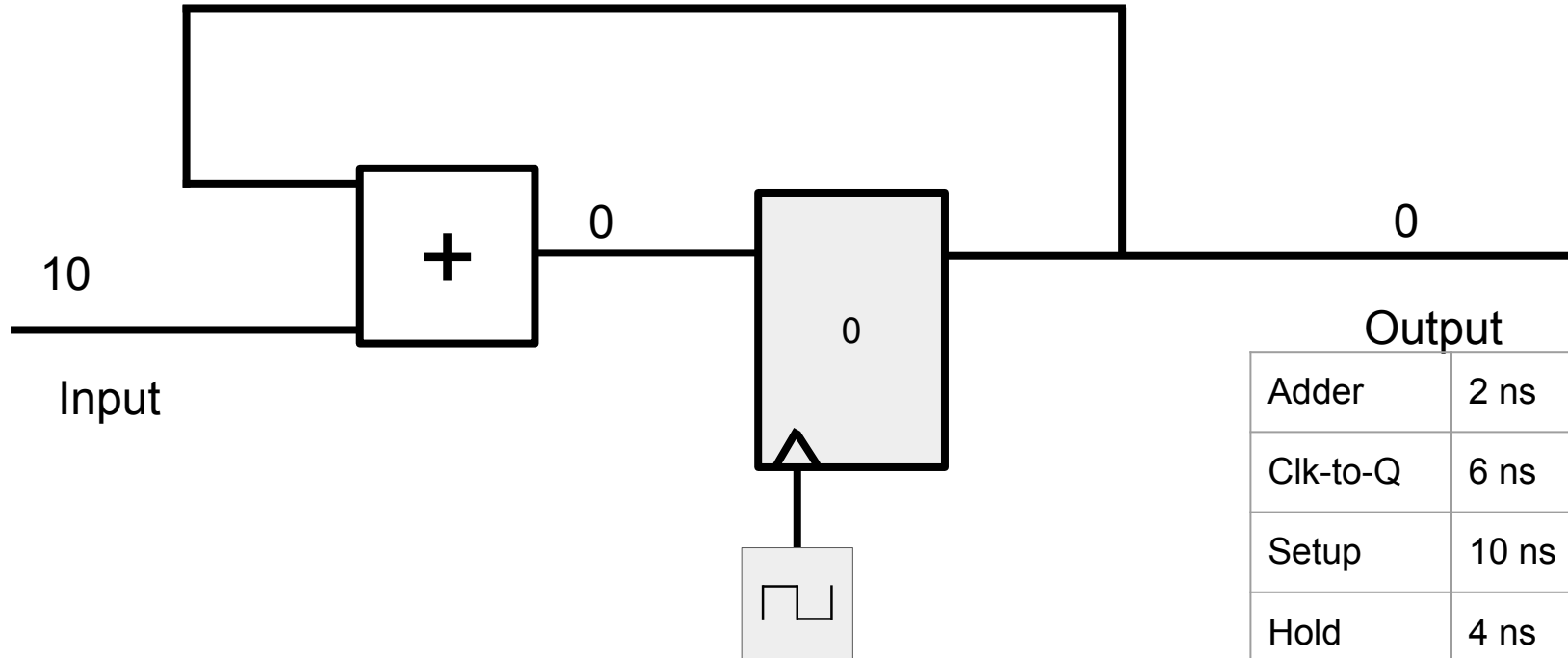
Min time to first register input change > Hold time



Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

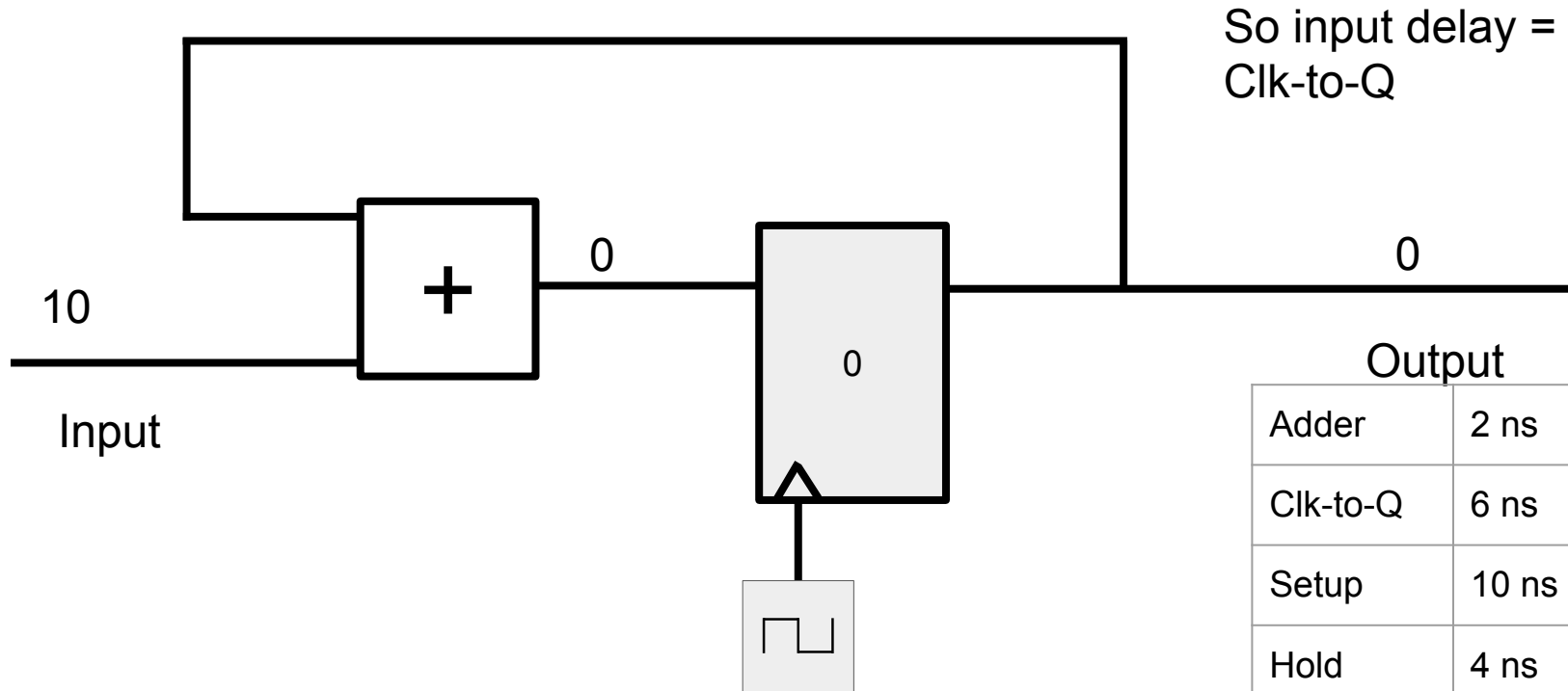
Input delay (relative to clk) + Min logic delay > Hold time



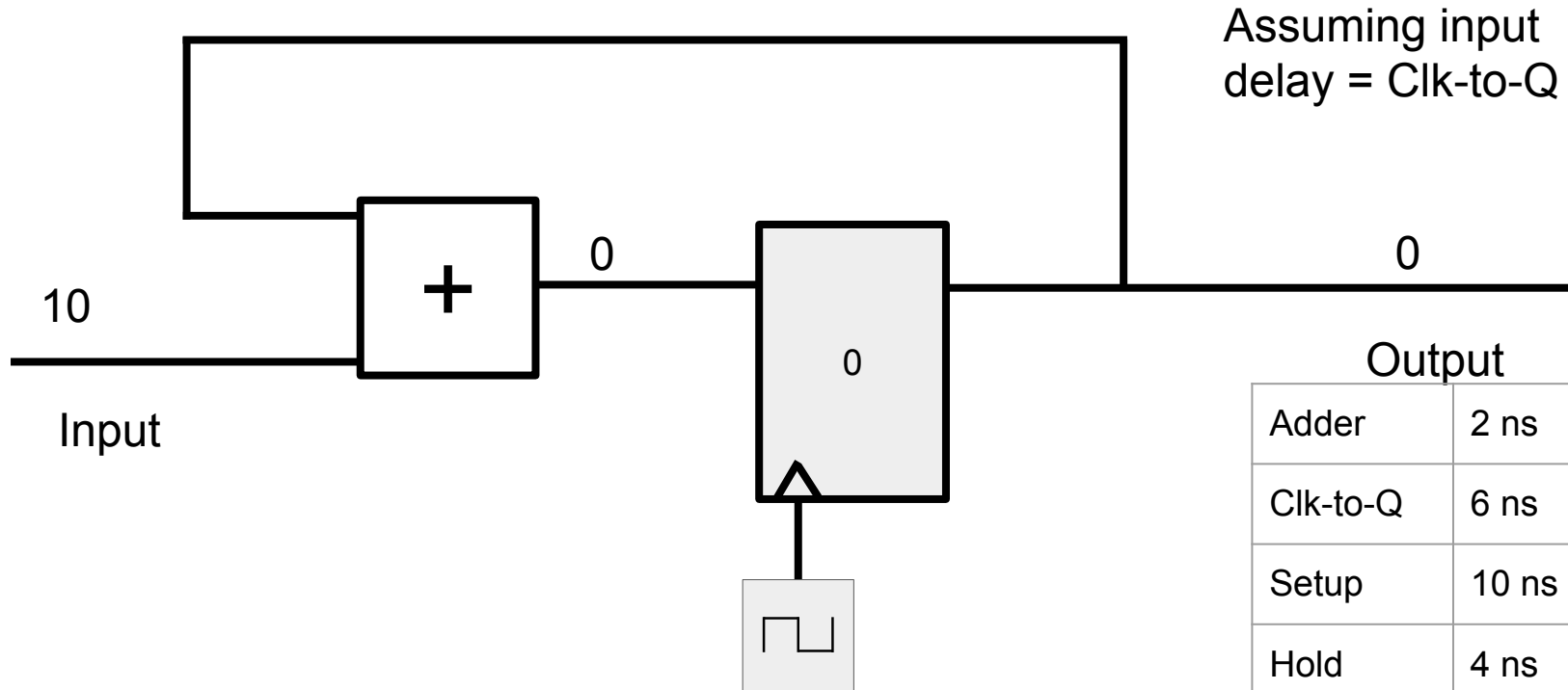
Output

Adder	2 ns
Clk-to-Q	6 ns
Setup	10 ns
Hold	4 ns

Common assumption: Inputs come from registers too



Therefore: Clk-to-Q + Min logic delay > Hold time

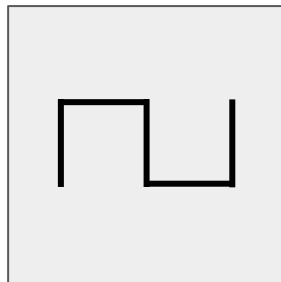


Clock Periods

Modern CPU clocks run at ~2 GHz

- SI note: $1 \text{ Hz} = 1 \text{ s}^{-1} = 1$ cycle per second
- $2 \text{ GHz} = 2 \text{ billion cycles / second} = 0.5 \text{ ns/cycle}$

Extremely fast: Light travels only 1 foot per ns!



What is the maximum clock frequency (in GHz)?

clk-to-q delay: 1ns
setup time: 1ns
hold time: 1ns
AND delay: 1ns
(Assume all unconnected inputs
come from some other registers.)

